

Agent 项目之外面试问题（详细版）

80 道主流概念题：Loop、Graph、Harness、Prompt、Context、Skills、MCP、Eval 与 AI 开发 workflow

本版不是简短背诵稿，而是详细备考稿。每题按「直接回答→机制/代码→执行步骤→边界与权衡→形象例子」展开；英文缩写和专有词在首次出现时紧跟括号解释。

阅读方法与章节

1-8	Agent 基础、Loop 与自主性
9-16	Graph、状态与可恢复编排
17-24	Harness、运行时与长任务
25-32	Prompt、结构化输出与安全
33-40	Context Engineering、记忆与 RAG
41-48	Skills、Tools、MCP 与可复用能力
49-56	多 Agent、A2A 与协作模式
57-64	Eval、Trace、Guardrails 与运行治理
65-72	AI 辅助开发工具与个人 workflow
73-80	主流 Agent 框架、交互形态与生产平台
65-80	AI 辅助开发、主流框架与平台设计

使用建议：先练每题的「直接回答」，再根据面试官追问展开后面的代码、流程和边界。不要一次背完整页，否则反而容易显得生硬。

Agent 基础、Loop 与自主性

1. 什么是 AI Agent? 它和普通 Chatbot、Workflow 有什么区别?

一、直接回答

我把 Agent（能够根据目标自主选择步骤和工具的软件智能体）理解成“能自己决定下一步并动手做事的模型系统”。普通 Chatbot 主要给答案；Workflow 按程序员提前写好的路线走；Agent 则会根据当前结果选择工具、调整路径并继续执行。所以 Agent 的核心不是聊天，而是模型、工具、状态和反馈循环。

二、原理拆解

- Chatbot 主要做输入到输出的对话映射，Workflow 按人预先写好的路径运行，Agent 则会根据中间结果选择下一步。真正区别不是“有没有 LLM”，而是路径是否在运行时动态决策。
- Agent 的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- 我把 Agent 理解成“能自己决定下一步并动手做事的模型系统”。
- 普通 Chatbot 主要给答案；
- Workflow 按程序员提前写好的路线走；
- Agent 则会根据当前结果选择工具、调整路径并继续执行。
- 所以 Agent 的核心不是聊天，而是模型、工具、状态和反馈循环。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

普通客服机器人只回答退款规则；退款 Agent 还能查订单、判断权限、提交退款并确认数据库状态

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

2. 你了解 Agent Loop 吗？请描述一次完整循环。

一、直接回答

Agent（能够根据目标自主选择步骤和工具的软件智能体）

Loop（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）

就是一个不断“看情况、做决定、执行、再看结果”的循环。模型先读取当前任务和状态，决定直接回答还是调用工具；工具执行后把结果返回；模型再根据新结果判断下一步。循环必须有明确出口，例如任务完成、需要人工确认、超过步骤或费用上限。

二、原理拆解

- 完整 Loop 至少要记录 observation、decision、action、tool（Agent 可以调用的具体程序能力）result、updated state（流程在当前时刻保存的数据和执行状态）和 stop reason。如果没有可验证反馈和退出条件，它只是重复生成，不是健康的 Agent Loop。
- Agent 的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- Agent Loop（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）（观察、决策、行动、反馈并继续或结束的循环）
就是一个不断“看情况、做决定、执行、再看结果”的循环。
- 模型先读取当前任务和状态，决定直接回答还是调用工具；
- 工具执行后把结果返回；
- 模型再根据新结果判断下一步。
- 循环必须有明确出口，例如任务完成、需要人工确认、超过步骤或费用上限。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

模型先调用天气工具，读到“下雨”后再决定调用日历工具改行程，这就是两轮有环境反馈的 Loop。

术语复习

Agent Loop: 观察、决策、行动、反馈并继续或结束的循环；Agent: 能够根据目标自主选择步骤和工具的软件智能体；State: 流程在当前时刻保存的数据和执行状态；Tool: Agent 可以调用的具体程序能力

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Agent Loop 的一轮



3. ReAct 是什么？它与 Agent Loop 有什么关系？

一、直接回答

ReAct 把 reasoning 与 acting

交替组织：模型基于当前观察决定动作，工具结果成为下一轮观察。

工程实现通常不暴露完整思维链，而是保留结构化的 Tool（Agent 可以调用的具体程序能力）call、observation、状态和简短决策摘要。ReAct 是 Loop 的一种常见策略；Loop 是更广义的运行机制，还包含权限、重试、预算、停止条件和持久化。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体）的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- ReAct 把 reasoning 与 acting 交替组织：模型基于当前观察决定动作，工具结果成为下一轮观察。
- 工程实现通常不暴露完整思维链，而是保留结构化的 Tool call、observation、状态和简短决策摘要。
- ReAct 是 Loop 的一种常见策略；
- Loop 是更广义的运行机制，还包含权限、重试、预算、停止条件和持久化。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

查航班时先搜索、再根据结果调整日期，工具结果就是下一轮 observation

术语复习

Tool: Agent 可以调用的具体程序能力；Agent: 能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

4. Agent Loop 的退出条件应该怎么设计？

一、直接回答

至少包括：任务完成；模型不再请求工具；需要用户确认；工具返回终止状态；达到 max_steps、deadline、token/cost budget；重复动作或无进展；策略违规；不可恢复异常。不能只依赖模型说“完成了”，应优先验证环境 outcome，例如文件是否生成、测试是否通过、订单是否真的写入数据库。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体）的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- 至少包括：任务完成；
- 模型不再请求工具；
- 需要用户确认；
- 工具返回终止状态；
- 达到 max_steps、deadline、token/cost budget；
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

代码 Agent 说“修好了”不算结束，测试通过、目标文件确实修改才算真实完成

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

5. 如何检测 Agent 陷入死循环或无效循环？

一、直接回答

记录每轮 action、arguments、observation 摘要和状态哈希。若连续多轮工具与参数相同、错误不变、目标指标无改善，或状态哈希周期性重复，就判定 stalled。处理方式是注入错误摘要、换策略/模型、回到最近 checkpoint（保存到某一步的任务状态快照）、请求人工输入或终止。还要设置硬性 step、时间和费用上限，避免检测器本身失效。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体）的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- 记录每轮 action、arguments、observation 摘要和状态哈希。
- 若连续多轮工具与参数相同、错误不变、目标指标无改善，或状态哈希周期性重复，就判定 stalled。
- 处理方式是注入错误摘要、换策略/模型、回到最近 checkpoint、请求人工输入或终止。
- 还要设置硬性 step、时间和费用上限，避免检测器本身失效。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

连续三次调用同一个 URL 都返回 403，继续调用没有意义，应换凭据或请求人工处理

术语复习

checkpoint：保存到某一步的任务状态快照；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

6. Agent 的 planning 有哪些方式？

一、直接回答

常见方式有即时 ReAct、先计划后执行（Plan-and-Execute）、分层任务树、图搜索，以及执行中动态重规划。短且可逆的任务适合即时决策；长任务适合先生成可检查的里程碑；外部环境变化大时需要滚动重规划。计划不是越详细越好，过早展开所有步骤会浪费 token，也会在环境变化后迅速失效。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体）的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- 常见方式有即时 ReAct、先计划后执行（Plan-and-Execute）、分层任务树、图搜索，以及执行中动态重规划。
- 短且可逆的任务适合即时决策；
- 长任务适合先生成可检查的里程碑；
- 外部环境变化大时需要滚动重规划。
- 计划不是越详细越好，过早展开所有步骤会浪费 token，也会在环境变化后迅速失效。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

改一个变量可直接 ReAct；跨 20 个文件重构则先列里程碑，再逐步执行

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

7. 什么时候不应该使用 Agent?

一、直接回答

路径固定、规则明确、低延迟高吞吐、结果必须完全确定的任务，普通函数、状态机或 Workflow 更合适。单次检索+生成能解决的问题也不必引入自主 Loop。Agent（能够根据目标自主选择步骤和工具的软件智能体）会增加延迟、成本、不可预测性和安全面；只有当步骤难以预先枚举、需要动态使用工具或处理开放环境时，它的收益才可能覆盖这些成本。

二、原理拆解

- Agent 的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- 路径固定、规则明确、低延迟高吞吐、结果必须完全确定的任务，普通函数、状态机或 Workflow 更合适。
- 单次检索+生成能解决的问题也不必引入自主 Loop。
- Agent 会增加延迟、成本、不可预测性和安全面；
- 只有当步骤难以预先枚举、需要动态使用工具或处理开放环境时，它的收益才可能覆盖这些成本。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

每天固定把 CSV 转 PDF 用脚本最合适，不需要让 Agent 每次重新规划

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

8. 怎样衡量一个 Agent 的自主性？自主性越高越好吗？

一、直接回答

可从目标拆解、工具选择、路径规划、错误恢复、运行时长和是否需要人工确认等维度衡量。自主性不是单一等级，也不是越高越好。高风险写操作应收紧权限并增加 HITL；只读研究任务可以放宽。成熟设计追求“与风险匹配的最小必要自主性”，而不是无限放权。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体）的基本组成是目标、模型、工具、状态、运行循环和退出条件。
- Loop 每轮应保留结构化观察、动作参数、工具结果和当前状态，不是无限追加聊天文本。

三、实际执行流程

- 可从目标拆解、工具选择、路径规划、错误恢复、运行时长和是否需要人工确认等维度衡量。
- 自主性不是单一等级，也不是越高越好。
- 高风险写操作应收紧权限并增加 HITL；
- 只读研究任务可以放宽。
- 成熟设计追求“与风险匹配的最小必要自主性”，而不是无限放权。
- 一轮标准流程是读取目标和环境→决定回答或调工具→执行→读取反馈→更新状态→判断继续或结束。
- 规划可以是一次性分步、边做边调整、树搜索或由规划者与执行者分工，选择取决于任务不确定性。

四、边界、异常和权衡

- 自主性越高，灵活性越高，同时错误面、成本和审计难度也越高。
- 明确、固定、高风险且可直接写规则的任务，通常更适合普通 Workflow，不必强行使用 Agent。

五、形象例子

读公开网页可自动完成；删除生产数据库必须停下来让人确认

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Graph、状态与可恢复编排

9. 你了解 Graph 吗？为什么 Agent 编排会使用图？

一、直接回答

Graph（用节点和连接关系表示任务流程的图结构）就是把复杂任务画成一张能运行的流程图。节点表示做一件事，边表示下一步去哪，State（流程在当前时刻保存的数据和执行状态）保存大家共同需要的数据。它适合有分支、循环、并行和暂停恢复的任务。简单两三步流程没必要强行上图，代码反而更直观。

二、原理拆解

- 使用图的核心原因是显式表达分支、循环、并行、汇合和恢复点。对于只有直线两步的任务，图反而会增加学习和调试成本。
- Graph 把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State。
- checkpoint（保存到某一步的任务状态快照）
保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- Graph 就是把复杂任务画成一张能运行的流程图。
- 节点表示做一件事，边表示下一步去哪，State 保存大家共同需要的数据。
- 它适合有分支、循环、并行和暂停恢复的任务。
- 简单两三步流程没必要强行上图，代码反而更直观。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

报销流程可画成“识别票据→校验金额→条件审批→入账”；金额大时走人工分支，中断后从 checkpoint 继续。

术语复习

Graph: 用节点和连接关系表示任务流程的图结构; State: 流程在当前时刻保存的数据和执行状态; Node: 图流程中的一个处理步骤; Edge: 连接两个步骤并决定流向的边; checkpoint: 保存到某一步的任务状态快照; fan-out: 把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Graph 编排



10. LangGraph 中 State、Node、Edge 分别是什么？

一、直接回答

State（流程在当前时刻保存的数据和执行状态）是跨节点传递的结构化数据，如 messages、plan、retrieved_docs、error 和 budget；Node（图流程中的一个处理步骤）是读取状态并返回增量更新的函数；Edge（连接两个步骤并决定流向的边）决定下一个节点，conditional edge 根据状态动态路由。Graph（用节点和连接关系表示任务流程的图结构）编译后由运行时执行。状态应保存原始结构化数据，而不是把一切拼成格式化文本，便于不同节点复用和调试。

二、原理拆解

- Graph 把处理步骤表示为 Node，把流转条件表示为 Edge，把跨步骤数据放进 State。
- checkpoint（保存到某一步的任务状态快照）
保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- State 是跨节点传递的结构化数据，如 messages、plan、retrieved_docs、error 和 budget；
- Node 是读取状态并返回增量更新的函数；
- Edge 决定下一个节点，conditional edge 根据状态动态路由。
- Graph 编译后由运行时执行。
- 状态应保存原始结构化数据，而不是把一切拼成格式化文本，便于不同节点复用和调试。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

State 保存订单号和审批状态，Node 做查单，Edge 根据金额决定自动批准还是人工复核

术语复习

Graph：用节点和连接关系表示任务流程的图结构；State：流程在当前时刻保存的数据和执行状态；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；checkpoint：保存到某一步的任务状态快照；fan-out：把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

11. Graph 与普通 Workflow/状态机有什么区别？

一、直接回答

它们并非对立。Graph（用节点和连接关系表示任务流程的图结构）是表达 Workflow 和状态机的一种形式；区别在于 Agent（能够根据目标自主选择步骤和工具的软件智能体）Graph 常把 LLM 决策作为条件边，并提供 checkpoint（保存到某一步的任务状态快照）、streaming、interrupt 和长期运行能力。传统状态机的状态与转移完全确定；Agent Graph 可以包含概率性节点，但关键边界仍应由确定性代码约束。

二、原理拆解

- Graph 把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State（流程在当前时刻保存的数据和执行状态）。
- checkpoint 保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- Graph 是表达 Workflow 和状态机的一种形式；
- 区别在于 Agent Graph 常把 LLM 决策作为条件边，并提供 checkpoint、streaming、interrupt 和长期运行能力。
- 传统状态机的状态与转移完全确定；
- Agent Graph 可以包含概率性节点，但关键边界仍应由确定性代码约束。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

普通状态机的边写死；Agent Graph 可让模型在“继续搜索”和“开始回答”之间选择

术语复习

checkpoint：保存到某一步的任务状态快照；Graph：用节点和连接关系表示任务流程的图结构；Agent：能够根据目标自主选择步骤和工具的软件智能体；State：流程在当前时刻保存的数据和执行状态；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；fan-out：把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

12. 什么是 durable execution?

一、直接回答

durable execution（任务中断后可从已保存状态继续执行）
指长任务在每个可靠边界保存状态，进程崩溃、网络中断或人工暂停后能从最近成功节点恢复，而不是从头运行。它要求 checkpoint（保存到某一步的任务状态快照）、稳定 thread/task ID、可序列化状态和幂等副作用。恢复时必须知道哪些步骤已提交、哪些可安全重放，这比简单把聊天记录存数据库更严格。

二、原理拆解

- Graph（用节点和连接关系表示任务流程的图结构）把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State（流程在当前时刻保存的数据和执行状态）。
- checkpoint 保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- durable execution 指长任务在每个可靠边界保存状态，进程崩溃、网络中断或人工暂停后能从最近成功节点恢复，而不是从头运行。
- 它要求 checkpoint、稳定 thread/task ID、可序列化状态和幂等副作用。
- 恢复时必须知道哪些步骤已提交、哪些可安全重放，这比简单把聊天记录存数据库更严格。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

任务跑到第 6 步进程崩溃，重启后从第 5 步 checkpoint 恢复，而不是重新付费跑前 5 步

术语复习

durable execution：任务中断后可从已保存状态继续执行；checkpoint：保存到某一步的任务状态快照；Graph：用节点和连接关系表示任务流程的图结构；State：流程在当前时刻保存的数据和执行状态；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；fan-out：把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

13. 为什么 Human-in-the-Loop 通常依赖 checkpoint?

一、直接回答

审批可能几分钟甚至几天后才发生。系统要在提出审批时保存图状态、待执行动作和上下文，然后释放进程资源；用户 approve/edit/reject 后再从同一节点恢复。没有持久化，只能一直占用内存或丢失现场。写操作发生在 interrupt 前时必须幂等，否则恢复会重复副作用。

二、原理拆解

- Graph（用节点和连接关系表示任务流程的图结构）把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State（流程在当前时刻保存的数据和执行状态）。
- checkpoint（保存到某一步的任务状态快照）
保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- 审批可能几分钟甚至几天后才发生。
- 系统要在提出审批时保存图状态、待执行动作和上下文，然后释放进程资源；
- 用户 approve/edit/reject 后再从同一节点恢复。
- 没有持久化，只能一直占用内存或丢失现场。
- 写操作发生在 interrupt 前时必须幂等，否则恢复会重复副作用。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

退款操作前暂停等待经理，第二天批准后从原节点继续，因此必须保存当时状态

术语复习

Graph：用节点和连接关系表示任务流程的图结构；State：流程在当前时刻保存的数据和执行状态；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；checkpoint：保存到某一步的任务状态快照；fan-out：把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

14. Graph 中如何实现 fan-out、join 和条件分支？

一、直接回答

fan-out（把一个任务拆给多个分支并行执行）

从一个节点把独立子任务分发到多个节点并行执行；join 等待 all 或 any

父节点完成后聚合结果；条件分支读取结构化状态选择下一条边。聚合时需要 reducer

解决并发更新冲突。并行只适合无依赖任务，存在数据依赖或共享写冲突时仍应串行或加事务。

二、原理拆解

- Graph（用节点和连接关系表示任务流程的图结构）把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State（流程在当前时刻保存的数据和执行状态）。
- checkpoint（保存到某一步的任务状态快照）
保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- fan-out 从一个节点把独立子任务分发到多个节点并行执行；
- join 等待 all 或 any 父节点完成后聚合结果；
- 条件分支读取结构化状态选择下一条边。
- 聚合时需要 reducer 解决并发更新冲突。
- 并行只适合无依赖任务，存在数据依赖或共享写冲突时仍应串行或加事务。
- 条件分支根据状态选边，fan-out 复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

同时让三个 Worker 查价格，join 节点等全部结果后选最低价

术语复习

fan-out：把一个任务拆给多个分支并行执行；Graph：用节点和连接关系表示任务流程的图结构；State：流程在当前时刻保存的数据和执行状态；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；checkpoint：保存到某一步的任务状态快照

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

15. Graph 中有循环时如何保证可终止？

一、直接回答

循环边必须配显式条件：质量达标、max_iterations、deadline、预算、无进展检测或人工中止。循环计数和最近错误应进入持久化 State（流程在当前时刻保存的数据和执行状态）（流程在当前时刻保存的数据和执行状态），不能只存在局部变量。编译阶段可检查没有出口的环；运行时还应设置全局上限，防止条件节点被模型错误判断。

二、原理拆解

- Graph（用节点和连接关系表示任务流程的图结构）把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State。
- checkpoint（保存到某一步的任务状态快照）
保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- 循环边必须配显式条件：质量达标、max_iterations、deadline、预算、无进展检测或人工中止。
- 循环计数和最近错误应进入持久化 State（流程在当前时刻保存的数据和执行状态），不能只存在局部变量。
- 编译阶段可检查没有出口的环；
- 运行时还应设置全局上限，防止条件节点被模型错误判断。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

自我改稿最多循环 3 次；第三次仍不达标就交给人工，而不是无限优化

术语复习

State：流程在当前时刻保存的数据和执行状态；Graph：用节点和连接关系表示任务流程的图结构；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；checkpoint：保存到某一步的任务状态快照；fan-out：把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

16. 什么时候选 LangGraph，什么时候自己写编排？

一、直接回答

需要持久化状态、暂停恢复、复杂分支、并行、子图和可视化时，LangGraph（用状态图组织可暂停、可恢复 Agent（能够根据目标自主选择步骤和工具的软件智能体）流程的框架）（用状态图组织可暂停、可恢复 Agent 流程的框架）能减少基础设施工作。只有几个固定步骤、同步执行且容易测试时，自定义 Python 编排更透明。选择标准不是流行度，而是状态复杂度、故障恢复要求、团队调试能力和框架锁定成本。

二、原理拆解

- Graph（用节点和连接关系表示任务流程的图结构）把处理步骤表示为 Node（图流程中的一个处理步骤），把流转条件表示为 Edge（连接两个步骤并决定流向的边），把跨步骤数据放进 State（流程在当前时刻保存的数据和执行状态）。
- checkpoint（保存到某一步的任务状态快照）
保存状态快照，使任务可以暂停等人确认、进程重启后继续或从错误步骤重试。

三、实际执行流程

- 需要持久化状态、暂停恢复、复杂分支、并行、子图和可视化时，LangGraph（用状态图组织可暂停、可恢复 Agent 流程的框架）能减少基础设施工作。
- 只有几个固定步骤、同步执行且容易测试时，自定义 Python 编排更透明。
- 选择标准不是流行度，而是状态复杂度、故障恢复要求、团队调试能力和框架锁定成本。
- 条件分支根据状态选边，fan-out（把一个任务拆给多个分支并行执行）复制子任务并行执行，join 等待指定分支后用 reducer 合并。
- 图中存在循环时，终止条件、轮次上限、时间/成本预算和无进展检测都应是状态的一部分。

四、边界、异常和权衡

- 不是所有流程都需要 Graph；只有固定两三步的同步任务，函数调用往往更清楚。
- 引入图框架之前要评估持久化、并发合并、节点幂等和迁移成本。

五、形象例子

只有“采集→总结”两步时自己写函数就够；需要暂停、恢复、并行和回放时再用 LangGraph。
三、Harness（为 Agent 提供循环、工具、状态和约束的运行外壳）、Runtime 与长任务工程

术语复习

LangGraph：用状态图组织可暂停、可恢复 Agent 流程的框架；Agent：能够根据目标自主选择步骤和工具的软件智能体；Graph：用节点和连接关系表示任务流程的图结构；State：流程在当前时刻保存的数据和执行状态；Node：图流程中的一个处理步骤；Edge：连接两个步骤并决定流向的边；checkpoint：保存到某一步的任务状态快照；fan-out：把一个任务拆给多个分支并行执行

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Harness、运行时与长任务

17. 你了解 Agent Harness 吗？它是什么？

一、直接回答

Harness（为 Agent（能够根据目标自主选择步骤和工具的软件智能体）提供循环、工具、状态和约束的运行外壳）（为 Agent 提供循环、工具、状态和约束的运行外壳）可以理解成 Agent 的“工作环境和管理系统”。模型只负责判断，Harness 负责把任务和上下文给模型、执行工具、保存状态、限制权限、处理循环和错误、记录日志。相同模型放在不同 Harness 里，实际能力可能差很多。

二、原理拆解

- Harness 的质量体现在它给模型看到哪些状态、提供什么工具、如何执行和验证、错了如何恢复。因此调优 Harness 往往比只换一个更大模型更有工程价值。
- Harness 不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- Harness（为 Agent 提供循环、工具、状态和约束的运行外壳）可以理解成 Agent 的“工作环境和管理系统”。
- 模型只负责判断，Harness 负责把任务和上下文给模型、执行工具、保存状态、限制权限、处理循环和错误、记录日志。
- 相同模型放在不同 Harness 里，实际能力可能差很多。
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）
强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

同一个模型放进普通聊天框只是助手；加入终端、测试、权限、状态和 Loop 后，才可以变成能长时间完成编码任务的 coding Agent。

术语复习

Harness: 为 Agent

提供循环、工具、状态和约束的运行外壳；Agent: 能够根据目标自主选择步骤和工具的软件智能体；application legibility: 让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Agent Harness



18. Model、Agent、Harness、Framework 四者如何区分？

一、直接回答

Model 负责生成与推理；Agent（能够根据目标自主选择步骤和工具的软件智能体）是模型加目标、工具、状态和决策循环形成的行为主体；Harness（为 Agent 提供循环、工具、状态和约束的运行外壳）是运行 Agent 的具体外壳；Framework 提供构建多个 Harness/Workflow 的抽象与组件。相同模型放进不同 Harness，表现可能差异很大，所以 Agent 评测实际测的是“模型+Harness+工具+环境”。

二、原理拆解

- Harness
不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- Model 负责生成与推理；
- Agent 是模型加目标、工具、状态和决策循环形成的行为主体；
- Harness 是运行 Agent 的具体外壳；
- Framework 提供构建多个 Harness/Workflow 的抽象与组件。
- 相同模型放进不同 Harness，表现可能差异很大，所以 Agent 评测实际测的是“模型+Harness+工具+环境”。
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）
强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

GPT 是模型，Codex 是带工具和运行时的 Agent 产品，LangGraph（用状态图组织可暂停、可恢复 Agent 流程的框架）是搭建运行流程的框架

术语复习

Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；Agent：能够根据目标自主选择步骤和工具的软件智能体；application legibility：让应用状态和动作对 Agent 清楚可读；LangGraph：用状态图组织可暂停、可恢复 Agent 流程的框架

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

19. 一个好的 coding-agent Harness 应包含什么？

一、直接回答

应包含代码库检索、shell/编辑器、测试和静态检查、Git diff、网络与文件权限、隔离工作区、任务计划、进度记录、失败恢复、日志/指标，以及最终自检。长任务还需要 checkpoint（保存到某一步的任务状态快照）、剩余工作清单、可重启开发环境和独立 review Agent（能够根据目标自主选择步骤和工具的软件智能体）。核心目标是让应用、测试和运行状态对 Agent（能够根据目标自主选择步骤和工具的软件智能体）可观察、可操作、可验证。

二、原理拆解

- Harness（为 Agent 提供循环、工具、状态和约束的运行外壳）
不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- 应包含代码库检索、shell/编辑器、测试和静态检查、Git diff、网络与文件权限、隔离工作区、任务计划、进度记录、失败恢复、日志/指标，以及最终自检。
- 长任务还需要 checkpoint、剩余工作清单、可重启开发环境和独立 review Agent（能够根据目标自主选择步骤和工具的软件智能体）。
- 核心目标是让应用、测试和运行状态对 Agent 可观察、可操作、可验证。
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）
强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

Agent 改完代码后必须自己运行 pytest、读取失败日志、修复并再次测试

术语复习

checkpoint：保存到某一步的任务状态快照；Agent：能够根据目标自主选择步骤和工具的软件智能体；Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；application legibility：让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

20. 为什么 Harness 会显著影响 Agent 能力？

一、直接回答

模型只能基于看到的上下文和工具反馈行动。若工具描述模糊、日志不可读、测试难启动、权限不清 或状态无法恢复，再强的模型也会猜测和重复劳动。好的 Harness（为 Agent（能够根据目标自主选择步骤和工具的软件智能体）提供循环、工具、状态和约束的运行外壳）（为 Agent 提供循环、工具、状态和约束的运行外壳）把环境“变得可读”，把动作做成高层、可验证工具，并在每轮提供最相关反馈，从而放大模型能力并控制错误半径。

二、原理拆解

- Harness
不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- 模型只能基于看到的上下文和工具反馈行动。
- 若工具描述模糊、日志不可读、测试难启动、权限不清 或状态无法恢复，再强的模型也会猜测和重复劳动。
- 好的 Harness（为 Agent 提供循环、工具、状态和约束的运行外壳）
把环境“变得可读”，把动作做成高层、可验证工具，并在每轮提供最相关反馈，从而放大模型能力并控制错误半径。
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）
强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

如果 Agent 看不到日志，它只能猜；把日志和指标暴露出来后，它才能像工程师一样定位

术语复习

Harness: 为 Agent 提供循环、工具、状态和约束的运行外壳；Agent: 能够根据目标自主选择步骤和工具的软件智能体；application legibility: 让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

21. 长时间 Agent 如何跨多个 context window 工作？

一、直接回答

不能无限保留原始消息。常见做法是维护任务说明、架构 约束、进度文件、决策日志、当前计划、未完成事项和测试状态；旧轨迹压缩成结构化摘要，重要产物写入文件或数据库；新窗口启动时先重新读取这些持久化状态，再检查真实环境。重点是外化记忆，而不是相信模型会“记住”。

二、原理拆解

- Harness（为 Agent（能够根据目标自主选择步骤和工具的软件智能体）提供循环、工具、状态和约束的运行外壳）不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- 不能无限保留原始消息。
- 常见做法是维护任务说明、架构 约束、进度文件、决策日志、当前计划、未完成事项和测试状态；
- 旧轨迹压缩成结构化摘要，重要产物写入文件或 数据库；
- 新窗口启动时先重新读取这些持久化状态，再检查真实环境。
- 重点是外化记忆，而不是相信模型会“记住”。
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

长任务把“已完成、失败原因、下一步”写进 progress.md，新窗口先读文件再继续

术语复习

Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；Agent：能够根据目标自主选择步骤和工具的软件智能体；application legibility：让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

22. 什么是 application legibility?

一、直接回答

指把应用状态以 Agent（能够根据目标自主选择步骤和工具的软件智能体）能读取和验证的形式暴露出来，例如可启动的独立实例、结构化日志、DOM /截图、指标、Trace（一次请求从输入到所有调用和输出的完整记录）（一次请求从输入到所有调用和输出的完整记录）、数据库检查工具和一键测试。它减少 Agent 对黑盒系统的猜测。Harness（为 Agent 提供循环、工具、状态和约束的运行外壳）Engineering 的重要方向不是只优化 Prompt（发给模型的任务指令和上下文），而是改造开发环境，让 Agent 能看到结果、定位错误并自我验证。

二、原理拆解

- Harness
不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- 指把应用状态以 Agent 能读取和验证的形式暴露出来，例如可启动的独立实例、结构化日志、DOM /截图、指标、Trace（一次请求从输入到所有调用和输出的完整记录）、数据库检查工具和一键测试。
- 它减少 Agent 对黑盒系统的猜测。
- Harness Engineering 的重要方向不是只优化 Prompt，而是改造开发环境，让 Agent 能看到结果、定位错误并自我验证。
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）
强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

让 Agent 能直接启动独立应用并看到 DOM，比让人截图后口头描述问题更可靠

术语复习

Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；Prompt：发给模型的任务指令和上下文；Trace：一次请求从输入到所有调用和输出的完整记录；Agent：能够根据目标自主选择步骤和工具的软件智能体；application legibility：让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

23. Agent 使用沙箱有什么意义？

一、直接回答

沙箱限制文件、网络、进程和凭据访问，降低误删、数据泄露和恶意网页提示词注入造成的爆炸半径。

。理想设计默认最小权限，按动作临时升级；工作目录隔离；网络域名 allowlist；密钥通过短期凭证注入；高风险命令需要审批；所有外部副作用可审计。

二、原理拆解

- Harness（为 Agent（能够根据目标自主选择步骤和工具的软件智能体）提供循环、工具、状态和约束的运行外壳）不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- 沙箱限制文件、网络、进程和凭据访问，降低误删、数据泄露和恶意网页提示词注入造成的爆炸半径。
- 理想设计默认最小权限，按动作临时升级；
- 网络域名 allowlist；
- 密钥通过短期凭证注入；
- 高风险命令需要审批；
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

网页里的恶意文字想让 Agent 删除文件，沙箱会因为目录权限不允许而阻止

术语复习

Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；Agent：能够根据目标自主选择步骤和工具的软件智能体；application legibility：让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

24. 如何设计可恢复的长任务 Harness?

一、直接回答

把任务拆成可提交里程碑；每步保存 checkpoint（保存到某一步的任务状态快照）、输入、输出、环境版本和幂等键；失败按错误分类重试；副作用采用事务或补偿；恢复前验证已有产物，而不是盲目重放；保留心跳和租约避免两个 Worker 同时接管；最终由独立验证器检查 outcome。

二、原理拆解

- Harness（为 Agent（能够根据目标自主选择步骤和工具的软件智能体）提供循环、工具、状态和约束的运行外壳）不是模型本身，而是把任务、上下文、工具、沙箱、记忆、限权、日志和循环组织起来的运行系统。
- 同一个模型放在不同 Harness 中，会因工具可见性、反馈质量和恢复能力不同而表现出很大差异。

三、实际执行流程

- 把任务拆成可提交里程碑；
- 每步保存 checkpoint、输入、输出、环境版本和幂等键；
- 失败按错误分类重试；
- 副作用采用事务或补偿；
- 恢复前验证已有产物，而不是盲目重放；
- 长任务要把目标、当前计划、已完成产物、未解决问题和快照持久化，而不是依赖聊天历史一直不丢。
- 沙箱限制文件、网络和系统权限，同时保留可销毁的执行环境和完整审计记录。

四、边界、异常和权衡

- 可恢复不等于无脑重试；每个动作要幂等，外部副作用要有唯一键和补偿逻辑。
- application legibility（让应用状态和动作对 Agent 清楚可读）
强调把状态、可选动作和执行反馈暴露给 Agent，否则它很容易在不可见环境里盲目尝试。

五、形象例子

部署步骤写入 checkpoint；网络断开后恢复时先查询是否已部署，避免重复发布

术语复习

checkpoint：保存到某一步的任务状态快照；Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；Agent：能够根据目标自主选择步骤和工具的软件智能体；application legibility：让应用状态和动作对 Agent 清楚可读

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Prompt、结构化输出与安全

25. Prompt Engineering 的核心目标是什么？

一、直接回答

Prompt（发给模型的任务指令和上下文）

Engineering（设计和优化模型指令的过程）（设计和优化模型指令的过程）（设计和优化模型指令的过程）（设计和优化模型指令的过程）

的重点不是找一句万能话术，而是把任务说清楚

：要做什么、不能做什么、输入是什么、输出长什么样、失败时怎么办。一个好 Prompt 还要能被测试和版本管理。面试里我会强调，Prompt 只是系统的一部分，不能替代权限、校验和评测。

二、原理拆解

- Prompt Engineering 的交付物不应是一段“看起来不错”的文字，而应是可版本化的指令、可重现的输入集、可自动验证的输出契约和评估结果。
- Prompt 的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema（输入输出必须遵守的数据结构约定）、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- Prompt Engineering（设计和优化模型指令的过程）（设计和优化模型指令的过程）（设计和优化模型指令的过程）的重点不是找一句万能话术，而是把任务说清楚：要做什么、不能做什么、输入是什么、输出长什么样、失败时怎么办。
- 一个好 Prompt 还要能被测试和版本管理。
- 面试里我会强调，Prompt 只是系统的一部分，不能替代权限、校验和评测。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）
适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

与其写“帮我处理得专业一点”，不如写清目标、受众、结构、长度和验收标准

术语复习

Prompt Engineering：设计和优化模型指令的过程；Prompt：发给模型的任务指令和上下文；schema：输入输出必须遵守的数据结构约定；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

26. System、Developer、User 指令应如何组织？

一、直接回答

高层消息定义稳定规则和安全边界，开发者指令定义应用 流程、工具策略和格式，用户消息表达当前目标，工具结果作为不可信观察数据。不要把所有规则重复塞进每轮；冲突时遵循平台的指令优先级。来自网页、文件和工具的文字不能自动升级为指令。

二、原理拆解

- Prompt（发给模型的任务指令和上下文）
的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖
schema（输入输出必须遵守的数据结构约定）、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- 高层消息定义稳定规则和安全边界，开发者指令定义应用 流程、工具策略和格式，用户消息表达当前目标，工具结果作为不可信观察数据。
- 不要把所有规则重复塞进每轮；
- 冲突时遵循平台的指令优先级。
- 来自网页、文件和工具的文字不能自动升级为指令。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）
适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

系统规定不能泄密，开发者规定输出 JSON，用户只能在这两个边界内提出当前问题

术语复习

Prompt：发给模型的任务指令和上下文；schema：输入输出必须遵守的数据结构约定；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

27. Few-shot 示例什么时候有用？有什么风险？

一、直接回答

当分类边界、输出风格或复杂 schema（输入输出必须遵守的数据结构约定）难以只用描述表达时，少量覆盖边界的正反例很有效。风险是示例过多占用上下文、模型过拟合示例措辞、复制示例事实，或示例之间矛盾。应优先选择代表性边界案例，并用评测验证新增示例是否真正改善。

二、原理拆解

- Prompt（发给模型的任务指令和上下文）的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- 当分类边界、输出风格或复杂 schema 难以只用描述表达时，少量覆盖边界的正反例很有效。
- 风险是 示例过多占用上下文、模型过拟合示例措辞、复制示例事实，或示例之间矛盾。
- 应优先选择代表性边界案例，并用评测验证新增示例是否真正改善。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

给一个正确和一个错误的工具调用示例，比堆十个相似正例更能说明边界

术语复习

schema：输入输出必须遵守的数据结构约定；Prompt：发给模型的任务指令和上下文；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

28. 如何让模型稳定输出结构化 JSON？

一、直接回答

优先使用模型原生 Structured Outputs/JSON（常用于系统之间传输结构化字段的文本格式） schema（输入输出必须遵守的数据结构约定）；其次在 Prompt（发给模型的任务指令和上下文）中给字段定义、枚举和示例。接收端必须做 schema 校验、类型转换和业务约束，失败时把最小错误反馈给模型有限重试。不要用脆弱正则从自然语言中猜 JSON，也不要因为语法合法就认为业务语义正确。

二、原理拆解

- Prompt 的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON”。

三、实际执行流程

- 优先使用模型原生 Structured Outputs/JSON schema；
- 其次在 Prompt 中给字段定义、枚举和示例。
- 接收端必须做 schema 校验、类型转换和业务约束，失败时把最小错误反馈给模型有限重试。
- 不要用脆弱正则从自然语言中猜 JSON，也不要因为语法合法就认为业务语义正确。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）
适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

字段 age 即使是合法 JSON 字符串，也要校验必须是整数且大于 0

术语复习

schema：输入输出必须遵守的数据结构约定；Prompt：发给模型的任务指令和上下文；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

29. Prompt Chaining 有什么价值？

一、直接回答

把复杂任务拆成分类、检索、生成、校验等较简单步骤，每步可设置确定性 gate，通常能提高可控性和可观察性。代价是延迟、费用和误差传播。适合子任务边界清晰且中间结果可验证的场景；如果多个步骤只是重复改写，可能不如一个高质量调用。

二、原理拆解

- Prompt（发给模型的任务指令和上下文）的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema（输入输出必须遵守的数据结构约定）、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- 把复杂任务拆成分类、检索、生成、校验等较简单步骤，每步可设置确定性 gate，通常能提高可控性和可观察性。
- 代价是延迟、费用和误差传播。
- 适合子任务边界清晰且中间结果可验证的场景；
- 如果多个步骤只是重复改写，可能不如一个高质量调用。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

先让模型提取事实，再让第二步写摘要，每一步都能单独检查

术语复习

Prompt：发给模型的任务指令和上下文；schema：输入输出必须遵守的数据结构约定；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

30. 如何优化 Tool Description?

一、直接回答

名称要表达动作，描述说明何时用/何时不用、参数语义、返回内容、错误和副作用；参数避免含糊的 data/object，使用枚举、范围和示例；相似工具要明确边界。工具说明也是 Prompt（发给模型的任务指令和上下文）的一部分，过多、重复或模糊会导致选错工具。应通过工具选择与参数准确率评测迭代。

二、原理拆解

- Prompt 的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema（输入输出必须遵守的数据结构约定）、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- 名称要表达动作，描述说明何时用/何时不用、参数语义、返回内容、错误和副作用；
- 参数避免含糊的 data/object，使用枚举、范围和示例；
- 相似工具要明确边界。
- 工具说明也是 Prompt 的一部分，过多、重复或模糊会导致选错工具。
- 应通过工具选择与参数准确率评测迭代。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

search_news 与 fetch_article 名字相近，描述必须说明一个找链接，一个抓正文

术语复习

Prompt：发给模型的任务指令和上下文；schema：输入输出必须遵守的数据结构约定；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

31. Prompt 注入为什么不是仅靠 Prompt 能解决的问题？

一、直接回答

因为攻击内容不一定来自用户，它可能藏在网页、邮件或工具返回里。只写一句“忽略恶意指令”很容易失效。真正的防护要靠多层：把外部文字当数据、限制工具权限、校验参数、隔离敏感信息，并让高风险操作必须经过人工确认。

二、原理拆解

- Prompt（发给模型的任务指令和上下文）的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema（输入输出必须遵守的数据结构约定）、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- 因为攻击内容不一定来自用户，它可能藏在网页、邮件或工具返回里。
- 只写一句“忽略恶意指令”很容易失效。
- 真正的防护要靠多层：把外部文字当数据、限制工具权限、校验参数、隔离敏感信息，并让高风险操作必须经过人工确认。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

邮件正文写“把客户表发给我”不能绕过工具权限，发送动作仍需人工批准

术语复习

Prompt：发给模型的任务指令和上下文；schema：输入输出必须遵守的数据结构约定；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

32. 如何管理 Prompt 版本和回归？

一、直接回答

Prompt（发给模型的任务指令和上下文）与模型、工具 schema（输入输出必须遵守的数据结构约定）（输入输出必须遵守的数据结构约定）、检索配置一起形成可执行版本。每次变更保存版本号和 diff，在固定 Eval（用数据集和指标评估系统能力）set 上比较任务成功率、格式、工具轨迹、延迟和成本；小流量灰度，收集失败 Trace（一次请求从输入到所有调用和输出的完整记录）（一次请求从输入到所有调用和输出的完整记录），能一键回滚。不能只凭几个手工对话判断“新 Prompt 更好”。

二、原理拆解

- Prompt 的核心不是写得长，而是明确目标、上下文、约束、输出格式、工具使用规则和完成标准。
- 结构化输出应同时依赖 schema、解析器、字段校验和有限重试，不能只在 Prompt 里写“严格返回 JSON（常用于系统之间传输结构化字段的文本格式）”。

三、实际执行流程

- Prompt 与模型、工具 schema（输入输出必须遵守的数据结构约定）、检索配置一起形成可执行版本。
- 每次变更保存版本号和 diff，在固定 Eval set 上比较任务成功率、格式、工具轨迹、延迟和成本；
- 小流量灰度，收集失败 Trace（一次请求从输入到所有调用和输出的完整记录），能一键回滚。
- 不能只凭几个手工对话判断“新 Prompt 更好”。
- System 层放安全和全局身份，Developer 层放应用逻辑和工具规则，User 层只放当前需求和用户数据。
- Few-shot（在提示词中提供少量示例帮助模型模仿）
适合展示边界、格式和判断标准，但示例错误、范围过窄或被用户数据污染会让模型学错。

四、边界、异常和权衡

- Prompt 注入是信任边界问题；解法还包括分离不可信内容、最小工具权限、参数校验和高风险审批。
- Prompt 变更应有版本号、评估集、对照实验、灰度和回滚，否则上线后无法说明质量变化来自哪里。

五、形象例子

Prompt v3 上线前用同一批 100 条样本和 v2 对比，发现工具准确率下降就回滚

术语复习

schema：输入输出必须遵守的数据结构约定；Prompt：发给模型的任务指令和上下文；Trace：一次请求从输入到所有调用和输出的完整记录；Eval：用数据集和指标评估系统能力；JSON：常用于系统之间传输结构化字段的文本格式；Few-shot：在提示词中提供少量示例帮助模型模仿

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Context Engineering、记忆与 RAG

33. Context Engineering 与 Prompt Engineering 有什么区别？

一、直接回答

Prompt（发给模型的任务指令和上下文）主要解决“这次应该怎么回答”；Context Engineering（系统性选择、组织和压缩模型上下文）解决“这次推理到底给模型看哪些信息”。除了 Prompt，工具说明、历史消息、检索文档、状态和 Skill（可复用的任务操作说明、脚本和配套资源）都会进入上下文。Agent（能够根据目标自主选择步骤和工具的软件智能体）跑得越久，信息越多，就越需要筛选、压缩和排序。

二、原理拆解

- Prompt 只是上下文的一部分。Context Engineering 还决定这一轮显示哪些工具、检索哪些资料、保留哪段历史、写入什么状态以及为输出留多少 token。
- Context Engineering 管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- Prompt 主要解决“这次应该怎么回答”；
- Context Engineering 解决“这次推理到底给模型看哪些信息”。
- 除了 Prompt，工具说明、历史消息、检索文档、状态和 Skill 都会进入上下文。
- Agent 跑得越久，信息越多，就越需要筛选、压缩和排序。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

Prompt 像厨师菜谱中的“本次做法”；Context 还包括菜谱、现有食材、顾客忌口、厨房状态和之前已经完成的步骤。

术语复习

Context Engineering：系统性选择、组织和压缩模型上下文；Prompt：发给模型的任务指令和上下文；Skill：可复用的任务操作说明、脚本和配套资源；Agent：能够根据目标自主选择步骤和工具的软件智能体；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Context Engineering



34. 为什么 context window 更大仍然需要上下文工程？

一、直接回答

长窗口会增加延迟和成本，且模型可能被陈旧、重复或中间位置的信息 干扰。更多 token 不等于更高 有效信息密度。应把最相关、最新、权威的信息放入当前窗口，其他内容外化存储并按需检索；目标是最大化每个 token 的决策价值，而不是填满窗口。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 长窗口会增加延迟和成本，且模型可能被陈旧、重复或中间位置的信息 干扰。
- 更多 token 不等于更高 有效信息密度。
- 应把最相关、最新、权威的信息放入当前窗口，其他内容外化存 储并按需检索；
- 目标是最大化每个 token 的决策价值，而不是填满窗口。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

把 200 页无关聊天塞进长窗口，反而可能让模型忽略当前问题

术语复习

Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

35. 如何分配上下文预算？

一、直接回答

先为不可压缩内容保留预算：系统安全规则、当前任务、关键 Tool（Agent 可以调用的具体程序能力） schema（输入输出必须遵守的数据结构约定）和输出格式；再给最新观察、必要历史和检索证据；剩余空间用于模型输出。为每类设置上限和优先级，超限时先去重、过滤低相关内容、摘要旧历史，最后才截断。还要为下一次工具结果预留空间。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 先为不可压缩内容保留预算：系统安全规则、当前任务、关键 Tool schema 和输出格式；
- 再给最新观察、必要历史和检索证据；
- 剩余空间用于模型输出。
- 为每类设置上限和优先级，超限时先去重、过滤低相关内容、摘要旧历史，最后才截断。
- 还要为下一次工具结果预留空间。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

给安全规则留固定预算，旧聊天先摘要，大工具结果只保留关键字段

术语复习

schema：输入输出必须遵守的数据结构约定；Tool：Agent 可以调用的具体程序能力；Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

36. 短期记忆、长期记忆、工作记忆如何区分？

一、直接回答

工作记忆是当前推理窗口中的活跃状态；短期记忆通常按 thread 保存近期消息和任务状态，可通过 checkpoint（保存到某一步的任务状态快照）恢复；长期记忆跨会话保存用户偏好、事实、经验或程序性规则。长期记忆不应每轮全部加载，而应按命名空间、权限和相关性检索。记忆写入也需要质量门槛，避免永久保存错误。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 工作记忆是当前推理窗口中的活跃状态；
- 短期记忆通常按 thread 保存近期消息和任务状态，可通过 checkpoint 恢复；
- 长期记忆跨会话保存用户偏好、事实、经验或程序性规则。
- 长期记忆不应每轮全部加载，而应按命名空间、权限和相关性检索。
- 记忆写入也需要质量门槛，避免永久保存错误。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

当前对话记录是短期记忆，用户长期偏好是长期记忆，正在算的计划是工作记忆

术语复习

checkpoint：保存到某一步的任务状态快照；Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

37. 上下文压缩有哪些方法？

一、直接回答

消息裁剪、相同结果去重、旧轨迹摘要、结构化 State（流程在当前时刻保存的数据和执行状态）、只保留工具结果关键字段、把大文件写入外部存储并保存引用、检索式记忆、分层摘要和按需加载 Skills。摘要要保留目标、约束、已完成工作、关键决策、错误和下一步，并定期用真实环境校验，防止摘要漂移。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 消息裁剪、相同结果去重、旧轨迹摘要、结构化 State、
只保留工具结果关键字段、把大文件写入外部存储并保存引用、检索式记忆、分层摘要和按需加载 Skills。
- 摘要要保留目标、约束、已完成工作、关键决策、错误和下一步，并定期用真实环境校验，防止摘要漂移。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

把 100 条工具日志压成“已尝试三种 API（软件之间按约定请求和返回数据的接口），均因 401 失败”，并保留原日志路径

术语复习

State：流程在当前时刻保存的数据和执行状态；Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答；API：软件之间按约定请求和返回数据的接口

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

38. 什么是 context poisoning/context pollution?

一、直接回答

错误、恶意或与任务无关的信息进入上下文后，会影响后续多轮决策，且可能被摘要和长期记忆放大。来源包括网页注入、错误工具输出、跨用户历史混入和过时记忆。防护方式是标注 provenance/可信级别、会话隔离、写记忆前验证、工具输出 schema（输入输出必须遵守的数据结构约定）、冲突检测和允许遗忘。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 错误、恶意或与任务无关的信息进入上下文后，会影响后续多轮决策，且可能被摘要和长期记忆放大。
- 来源包括网页注入、错误工具输出、跨用户历史混入和过时记忆。
- 防护方式是标注 provenance/可信级别、会话隔离、写记忆前验证、工具输出 schema、冲突检测和允许遗忘。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

错误价格被写入长期记忆后会反复污染报价，所以写记忆前需要来源和校验

术语复习

schema：输入输出必须遵守的数据结构约定；Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

39. RAG 检索结果怎样放入上下文更有效？

一、直接回答

不要直接塞大量全文。先按权限、时间和元数据过滤，再混合召回与重排；保留可引用片段、来源和分数；按问题组织，而非按数据库顺序；对冲突证据同时呈现；控制 topK 和每段长度。生成答案时要求引用证据，并在证据不足时明确拒答或继续检索。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 不要直接塞大量全文。
- 先按权限、时间和元数据过滤，再混合召回与重排；
- 保留可引用片段、来源和分数；
- 按问题组织，而非按数据库顺序；
- 对冲突证据同时呈现；
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

问合同违约金时只放相关条款、页码和来源，不把整本合同全部塞进去

术语复习

Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

40. 如何判断上下文工程是否有效？

一、直接回答

观察任务成功率、工具选择准确率、引用正确率、上下文 token、首 token/总延迟、费用、无关信息比例和跨轮错误传播。做 context ablation：删除某类上下文看性能变化；比较全文、摘要、检索和不同排序；分析 attention 位置相关失败。最终用任务 outcome，而不是“Prompt（发给模型的任务指令和上下文）看起来很完整”评价。

二、原理拆解

- Context Engineering（系统性选择、组织和压缩模型上下文）
管的是模型这一轮到底看到什么：指令、工具定义、历史、检索证据、当前状态和输出预算都在范围内。
- 大 context window（模型一次能够读取的上下文容量）
只增加容量，不保证模型能找到关键证据；无关信息、冲突指令和旧状态仍会造成干扰。

三、实际执行流程

- 观察任务成功率、工具选择准确率、引用正确率、上下文 token、首 token/总延迟、费用、无关信息比例和跨轮错误传播。
- 做 context ablation：删除某类上下文看性能变化；
- 比较全文、摘要、检索和不同排序；
- 分析 attention 位置相关失败。
- 最终用任务 outcome，而不是“Prompt 看起来很完整”评价。
- 上下文预算应优先保留安全规则、当前目标、必需工具、相关证据和最近决策，再根据任务分配历史和示例。
- 长期记忆保存稳定偏好和事实，工作记忆保存当前计划与中间产物，短期记忆保存最近交互。

四、边界、异常和权衡

- 压缩不应只做摘要，还可以做结构化状态、检索式回填、工具轨迹去冗余和证据分组。
- RAG（检索增强生成，先找资料再让模型基于资料回答）
证据应标明来源、时间、权限和相关性，保留可引用片段，不应将整个文档无差别塞给模型。

五、形象例子

删掉历史摘要后准确率不变，说明它占 token 但没贡献，可以移出上下文

术语复习

Prompt：发给模型的任务指令和上下文；Context Engineering：系统性选择、组织和压缩模型上下文；context window：模型一次能够读取的上下文容量；RAG：检索增强生成，先找资料再让模型基于资料回答

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Skills、Tools、MCP 与可复用能力

41. 你觉得 Agent Skills 是什么？

一、直接回答

Skill（可复用的任务操作说明、脚本和配套资源）
不是一个按钮，而是一套可以重复使用的做事方法。它会告诉 Agent（能够根据目标自主选择步骤和工具的软件智能体）什么时候适用、要读什么资料、按什么步骤执行、用哪些脚本，以及最后怎样验收。把团队经验写成 Skill 后，下次不需要重新解释整套流程。

二、原理拆解

- Skill 描述“完成一类任务的方法”，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- Skill 不是一个按钮，而是一套可以重复使用的做事方法。
- 它会告诉 Agent 什么时候适用、要读什么资料、按什么步骤执行、用哪些脚本，以及最后怎样验收。
- 把团队经验写成 Skill 后，下次不需要重新解释整套流程。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt（发给模型的任务指令和上下文）用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent 暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

PDF Skill 不只是“会生成 PDF”，还规定字体、渲染检查、临时文件和交付格式

术语复习

Skill: 可复用的任务操作说明、脚本和配套资源; Agent: 能够根据目标自主选择步骤和工具的软件智能体; Plugin: 把技能、工具或外部服务打包接入系统的扩展; Tool: Agent 可以调用的具体程序能力; MCP: 模型上下文协议, 用于统一连接 Agent 与工具或数据; Prompt: 发给模型的任务指令和上下文; MCP Server: 按 MCP 协议向 Agent 暴露工具或资源的服务

六、面试口述建议

面试时不需要一次把所有术语倒出来; 先讲清原理和适用边界, 面试官追问时再展开工程细节。

42. Skill、Prompt、Tool、MCP Server、Plugin 有什么区别？

一、直接回答

我用一句话区分：Prompt（发给模型的任务指令和上下文）是本次指令，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是连接工具的协议，Skill（可复用的任务操作说明、脚本和配套资源）是一整套可复用流程，Plugin（把技能、工具或外部服务打包接入系统的扩展）是把这些能力打包安装和分发。它们可能组合使用，但职责不同。

二、原理拆解

- 可以用一句话区分：Prompt 是这次指令，Tool 是一个动作，Skill 是一套做事方法，MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）是按协议对外提供工具/资源的服务，Plugin 是可安装的打包形式。
- Skill 描述“完成一类任务的方法”，Tool 是具体动作，MCP 是暴露工具和资源的协议，Plugin 是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- 我用一句话区分：Prompt 是本次指令，Tool 是具体动作，MCP 是连接工具的协议，Skill 是一整套可复用流程，Plugin 是把这些能力打包安装和分发。
- 它们可能组合使用，但职责不同。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt 用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server 都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

Tool 是“执行导出 PDF”的按钮，Skill 是从读材料、排版、渲染到验收的整套操作手册，MCP Server 则是把导出工具按统一协议提供出来的服务。

术语复习

Prompt: 发给模型的任务指令和上下文; Plugin: 把技能、工具或外部服务打包接入系统的扩展; Skill: 可复用的任务操作说明、脚本和配套资源; Tool: Agent 可以调用的具体程序能力; MCP: 模型上下文协议，用于统一连接 Agent 与工具或数据; MCP Server: 按 MCP 协议向 Agent 暴露工具或资源的服务; Agent: 能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

能力层次



43. 为什么 Skill 要采用渐进式披露？

一、直接回答

如果启动时把所有 Skill（可复用的任务操作说明、脚本和配套资源）全文和资源塞进上下文，会迅速耗尽 token，并干扰工具选择。渐进式披露先暴露短名称和描述；任务匹配后再读取完整 SKILL.md；只有步骤需要时才加载具体参考、模板或脚本。这样兼顾可发现性、上下文效率和专业细节。

二、原理拆解

- Skill 描述“完成一类任务的方法”，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- 如果启动时把所有 Skill 全文和资源塞进上下文，会迅速耗尽 token，并干扰工具选择。
- 渐进式披露先暴露短名称和描述；
- 任务匹配后再读取完整 SKILL.md；
- 只有步骤需要时才加载具体参考、模板或脚本。
- 这样兼顾可发现性、上下文效率和专业细节。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt（发给模型的任务指令和上下文）用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

启动时只看到 Skill 名称；确认要做 PDF 后才加载完整 SKILL.md

术语复习

Skill：可复用的任务操作说明、脚本和配套资源；Plugin：把技能、工具或外部服务打包接入系统的扩展；Tool：Agent 可以调用的具体程序能力；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；Prompt：发给模型的任务指令和上下文；MCP Server：按 MCP 协议向 Agent 暴露工具或资源的服务；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

44. 一个高质量 SKILL.md 应包含什么？

一、直接回答

清晰触发范围、输入与前置条件、逐步流程、工具/脚本使用方式、输出验收标准、错误和回退、安全边界、需要读取的资源，以及最小示例。要区分硬性规则和建议，避免泛化成“尽量做好”。Skill（可复用的任务操作说明、脚本和配套资源）应可测试、可版本化，并允许用户指令覆盖非安全性默认值。

二、原理拆解

- Skill 描述“完成一类任务的方法”，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- 清晰触发范围、输入与前置条件、逐步流程、工具/脚本使用方式、输出验收标准、错误和回退、安全边界、需要读取的资源，以及最小示例。
- 要区分硬性规则和建议，避免泛化成“尽量做好”。
- Skill 应可测试、可版本化，并允许用户指令覆盖非安全性默认值。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt（发给模型的任务指令和上下文）用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

好的 Skill 会写明输入文件、执行步骤、失败处理和最终必须渲染检查

术语复习

Skill：可复用的任务操作说明、脚本和配套资源；Plugin：把技能、工具或外部服务打包接入系统的扩展；Tool：Agent 可以调用的具体程序能力；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；Prompt：发给模型的任务指令和上下文；MCP Server：按 MCP 协议向 Agent 暴露工具或资源的服务；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

45. 你常用或会设计哪些 Skills?

一、直接回答

开发场景常见：代码库探索、测试与修复、代码审查、API（软件之间按约定请求和返回数据的接口）（软件之间按约定请求和返回数据的接口）文档查询、数据库迁移、部署、日志排障、浏览器 UI 验证；知识工作常见：PDF/Word/PPT/表格生成、研究与引用、图像生成、会议纪要、项目管理。回答时最好挑 3-5 个讲清输入、工具、流程和验收，不要只罗列名称。

二、原理拆解

- Skill（可复用的任务操作说明、脚本和配套资源）描述“完成一类任务的方法”，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- 开发场景常见：代码库探索、测试与修复、代码审查、API（软件之间按约定请求和返回数据的接口）文档查询、数据库迁移、部署、日志排障、浏览器 UI 验证；
- 知识工作常见：PDF/Word/PPT/表格生成、研究与引用、图像生成、会议纪要、项目管理。
- 回答时最好挑 3-5 个讲清输入、工具、流程和验收，不要只罗列名称。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt（发给模型的任务指令和上下文）用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

代码审查 Skill 可固定执行 diff→测试→安全检查→按优先级输出问题

术语复习

API：软件之间按约定请求和返回数据的接口；Plugin：把技能、工具或外部服务打包接入系统的扩展；Skill：可复用的任务操作说明、脚本和配套资源；Tool：Agent 可以调用的具体程序能力；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；Prompt：发给模型的任务指令和上下文；MCP Server：按 MCP 协议向 Agent 暴露工具或资源的服务；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

46. Skill 如何自动选择？如何防止选错？

一、直接回答

选择器主要依据 Skill（可复用的任务操作说明、脚本和配套资源）名称、描述、任务语义和环境能力。描述要互斥且具体；高风险 Skill 不应仅靠语义自动触发；冲突时按优先级或向用户澄清。通过触发/不触发样本评测 precision、recall，并记录实际加载轨迹。Skill 内部仍要再次验证前置条件。

二、原理拆解

- Skill 描述“完成一类任务的方法”，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- 选择器主要依据 Skill 名称、描述、任务语义和环境能力。
- 描述要互斥且具体；
- 高风险 Skill 不应仅靠语义自动触发；
- 冲突时按优先级或向用户澄清。
- 通过触发/不触发样本评测 precision、recall，并记录实际加载轨迹。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt（发给模型的任务指令和上下文）用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

“做表格”触发表格 Skill，“控制已打开 Excel”触发另一个 Skill，描述必须能区分

术语复习

Skill：可复用的任务操作说明、脚本和配套资源；Plugin：把技能、工具或外部服务打包接入系统的扩展；Tool：Agent 可以调用的具体程序能力；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；Prompt：发给模型的任务指令和上下文；MCP Server：按 MCP 协议向 Agent 暴露工具或资源的服务；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

47. Skill 会带来哪些安全风险？

一、直接回答

Skill（可复用的任务操作说明、脚本和配套资源）

可能包含恶意指令、脚本、外部链接或过宽权限；资源还可能过期。应校验来源与签名、审查脚本、固定版本、限制文件/网络权限、显示其将执行的动作、对敏感步骤审批，并把 Skill 内容视为低于系统安全策略。安装和更新都应有审计。

二、原理拆解

- Skill 描述“完成一类任务的方法”，Tool（Agent 可以调用的具体程序能力）是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- Skill 可能包含恶意指令、脚本、外部链接或过宽权限；
- 资源还可能过期。
- 应校验来源与签名、审查脚本、固定版本、限制文件/网络权限、显示其将执行的动作、对敏感步骤审批，并把 Skill 内容视为低于系统安全策略。
- 安装和更新都应有审计。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt（发给模型的任务指令和上下文）用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

来路不明的 Skill 里如果藏有上传密钥脚本，安装审查和沙箱应阻止它

术语复习

Skill：可复用的任务操作说明、脚本和配套资源；Plugin：把技能、工具或外部服务打包接入系统的扩展；Tool：Agent 可以调用的具体程序能力；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；Prompt：发给模型的任务指令和上下文；MCP Server：按 MCP 协议向 Agent 暴露工具或资源的服务；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

48. MCP 的 Tools、Resources、Prompts 如何选择？

一、直接回答

需要执行动作或产生副作用时用 Tool（Agent 可以调用的具体程序能力）；需要读取可寻址数据时用 Resource；希望给用户提供参数化模板/工作流入口时用 Prompt（发给模型的任务指令和上下文）。不要把纯读取一律包装成高权限工具，也不要把复杂写操作伪装成资源。Host 还要根据能力协商、用户同意和权限策略决定是否暴露或调用。

二、原理拆解

- Skill（可复用的任务操作说明、脚本和配套资源）描述“完成一类任务的方法”，Tool 是具体动作，MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）是暴露工具和资源的协议，Plugin（把技能、工具或外部服务打包接入系统的扩展）是可安装的能力包。
- 高质量 SKILL.md 应写明触发条件、非适用场景、步骤、安全边界、资源路径、验证标准和失败回退。

三、实际执行流程

- 需要执行动作或产生副作用时用 Tool；
- 需要读取可寻址数据时用 Resource；
- 希望给用户提供参数化模板/工作流入口时用 Prompt。
- 不要把纯读取一律包装成高权限工具，也不要把复杂写操作伪装成资源。
- Host 还要根据能力协商、用户同意和权限策略决定是否暴露或调用。
- 渐进式披露是先给模型看 Skill 名称和短描述，只有选中后才读完整指令和资源，以节省上下文。
- MCP 中 Tool 用于执行，Resource 用于读取可寻址数据，Prompt 用于暴露可参数化的任务模板。

四、边界、异常和权衡

- Skill 自动选择需要正反样本、描述去重和冲突规则，高风险 Skill 不能仅依赖语义相似度自动触发。
- 第三方 Skill 和 MCP Server（按 MCP 协议向 Agent（能够根据目标自主选择步骤和工具的软件智能体）暴露工具或资源的服务）都是供应链输入，应固定版本、检查来源、限制权限并记录实际执行。

五、形象例子

查文件内容适合 Resource，执行删除适合 Tool，给用户提供模板入口适合 Prompt。七、多 Agent、协议与协作模式

术语复习

Prompt: 发给模型的任务指令和上下文; Tool: Agent 可以调用的具体程序能力; Plugin: 把技能、工具或外部服务打包接入系统的扩展; Skill: 可复用的任务操作说明、脚本和配套资源; MCP: 模型上下文协议, 用于统一连接 Agent 与工具或数据; MCP Server: 按 MCP 协议向 Agent 暴露工具或资源的服务; Agent: 能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来; 先讲清原理和适用边界, 面试官追问时再展开工程细节。

多 Agent、A2A 与协作模式

49. 多 Agent 常见协作模式有哪些？

一、直接回答

多 Agent（能够根据目标自主选择步骤和工具的软件智能体）有几种常见组织方式：主 Agent 统一派活；按类别路由到专家；多个 Worker 并行后汇总；一个生成、一个评审反复改进；或者专家之间直接 handoff。没有哪种一定最好，主要看任务是否可预先拆分、是否需要中心控制和能否并行。

二、原理拆解

- 选协作模式时先问三个问题：任务能否预拆？子任务能否并行？是否需要中央状态和最终裁决？这三个问题比追求某个热门术语更重要。
- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- 多 Agent 有几种常见组织方式：主 Agent 统一派活；
- 按类别路由到专家；
- 多个 Worker 并行后汇总；
- 一个生成、一个评审反复改进；
- 或者专家之间直接 handoff。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema（输入输出必须遵守的数据结构约定）、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

主 Agent 把查网页、查数据库和写报告分给三个 Worker，最后再由 Reviewer 核验引用和数字，这就是 Orchestrator-Workers（主调度者拆任务、多个执行者并行完成的协作模式）加评审的组合。

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务；schema：输入输出必须遵守的数据结构约定；Orchestrator-Workers：主调度者拆任务、多个执行者并行完成的协作模式

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

多 Agent 组织



50. 什么时候多 Agent 优于单 Agent?

一、直接回答

当子任务需要不同工具/权限/模型/上下文，能并行处理，或需要独立审查时，多 Agent（能够根据目标自主选择步骤和工具的软件智能体）有价值。若只是把同一 Prompt（发给模型的任务指令和上下文）拆成多个角色，往往增加延迟、token 和错误传播。应先尝试单 Agent+清晰工具，在评测证明工具过多、上下文冲突或专业分工确有收益后再拆。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- 当子任务需要不同工具/权限/模型/上下文，能并行处理，或需要独立审查时，多 Agent 有价值。
- 若只是把同一 Prompt 拆成多个角色，往往增加延迟、token 和错误传播。
- 应先尝试单 Agent+清晰工具，在评测证明工具过多、上下文冲突或专业分工确有收益后再拆。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema（输入输出必须遵守的数据结构约定）、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

单 Agent 有 5 个清晰工具时没必要拆；当财务和法律权限不同，拆专家 Agent 才有意义

术语复习

Prompt：发给模型的任务指令和上下文；Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务；schema：输入输出必须遵守的数据结构约定

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

51. Orchestrator-Workers 与 Swarm 有什么区别？

一、直接回答

Orchestrator 由中心节点拆解、分派和汇总，便于全局预算、权限和观测；Swarm 由当前 Agent（能够根据目标自主选择步骤和工具的软件智能体）通过 handoff 局部决定下一个专家，灵活且去中心化，但共享上下文容易膨胀，终止与责任归属更难。生产系统常混合：中心控制大任务，局部团队内部 handoff。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- Orchestrator 由中心节点拆解、分派和汇总，便于全局预算、权限和观测；
- Swarm 由当前 Agent 通过 handoff 局部决定下一个专家，灵活且去中心化，但共享上下文容易膨胀，终止与责任归属更难。
- 生产系统常混合：中心控制大任务，局部团队内部 handoff。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema（输入输出必须遵守的数据结构约定）、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

中心经理统一派活是 Orchestrator；专家之间自己转接更像 Swarm

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务；schema：输入输出必须遵守的数据结构约定

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

52. A2A 与 MCP 为什么互补？

一、直接回答

MCP（模型上下文协议，用于统一连接 Agent 与工具或数据） 标准化 LLM 应用如何连接工具、资源和 Prompt（发给模型的任务指令和上下文）； A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务） 标准化独立、可能不透明的 Agent（能够根据目标自主选择步骤和工具的软件智能体） 如何发现彼此、委派任务和交换结果。一个远程 Agent 可以内部使用 MCP 工具，却不暴露自己的内部记忆和实现。A2A 不是 Agent Framework，也不是 MCP 的替代品。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent handoff。
- A2A 解决 Agent 身份、能力发现、任务委派和结果交付； MCP 解决 Agent 如何使用工具和数据。

三、实际执行流程

- MCP 标准化 LLM 应用如何连接工具、资源和 Prompt；
- A2A 标准化独立、可能不透明的 Agent 如何发现彼此、委派任务和交换结果。
- 一个远程 Agent 可以内部使用 MCP 工具，却不暴露自己的内部记忆和实现。
- A2A 不是 Agent Framework，也不是 MCP 的替代品。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema（输入输出必须遵守的数据结构约定）、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

A2A 找远程翻译 Agent，翻译 Agent 内部再通过 MCP 调术语库

术语复习

Prompt：发给模型的任务指令和上下文； Agent：能够根据目标自主选择步骤和工具的软件智能体； MCP：模型上下文协议，用于统一连接 Agent 与工具或数据； A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务； schema：输入输出必须遵守的数据结构约定

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

53. Agent Card/Skill Card 应包含哪些信息？

一、直接回答

名称、描述、版本、服务 URL、认证与传输、能力、输入/输出模式和 Skills。Skill（可复用的任务操作说明、脚本和配套资源）要用稳定 id、明确 description、tags 和典型 example 帮助发现；但真实契约必须有 schema（输入输出必须遵守的数据结构约定）、错误语义和版本兼容规则。Card 是能力广告，不等于授权，客户端仍需验证身份和权限。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent（能够根据目标自主选择步骤和工具的软件智能体）handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- 名称、描述、版本、服务 URL、认证与传输、能力、输入/输出模式和 Skills。
- Skill 要用稳定 id、明确 description、tags 和典型 example 帮助发现；
- 但真实契约必须有 schema、错误语义和版本兼容规则。
- Card 是能力广告，不等于授权，客户端仍需验证身份和权限。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

Card 写“能订票”还不够，还要写输入城市、日期和返回格式

术语复习

schema：输入输出必须遵守的数据结构约定；Skill：可复用的任务操作说明、脚本和配套资源；Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

54. 多 Agent 如何防止上下文污染？

一、直接回答

默认按任务传最小结构化上下文，而不是共享所有历史；每条信息携带来源、时间和可信级别；按 tenant/user/thread 隔离；对返回做 schema（输入输出必须遵守的数据结构约定）验证；摘要由中心生成并允许核对原始证据；敏感字段脱敏。共享黑板模式需要访问控制和版本冲突策略。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent（能够根据目标自主选择步骤和工具的软件智能体）handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- 默认按任务传最小结构化上下文，而不是共享所有历史；
- 每条信息携带来源、时间和可信级别；
- 按 tenant/user/thread 隔离；
- 对返回做 schema 验证；
- 摘要由中心生成并允许核对原始证据；
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

财务 Agent 只收到金额和发票，不应看到用户与医疗 Agent 的完整对话

术语复习

schema：输入输出必须遵守的数据结构约定；Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

55. 多 Agent 任务失败如何做补偿？

一、直接回答

先定义任务状态和幂等键。纯计算步骤可重试或换 Agent（能够根据目标自主选择步骤和工具的软件智能体）；外部写操作采用 saga：每步记录提交结果和补偿动作；聚合节点区分必需与可选子任务，允许 partial success；达到阈值后交给人工。不能因为 Worker 超时就假设未执行，必须查询真实外部状态。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- 先定义任务状态和幂等键。
- 纯计算步骤可重试或换 Agent；
- 外部写操作采用 saga：每步记录提交结果和补偿动作；
- 聚合节点区分必需与可选子任务，允许 partial success；
- 达到阈值后交给人工。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema（输入输出必须遵守的数据结构约定）、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

扣款成功但发券失败时，系统要补发券或退款，而不是简单重跑整条链

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务；schema：输入输出必须遵守的数据结构约定

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

56. 如何评估多 Agent 系统是否真的更好？

一、直接回答

与单 Agent（能够根据目标自主选择步骤和工具的软件智能体）/固定 Workflow 建 baseline，比较端到端成功率、延迟、token/成本、工具错误、协调开销和稳定性。还要评估 delegation accuracy、每个 Worker 成功率、消息质量、无效 handoff、并行收益和失败传播。若准确率提升很小但成本翻倍，就应回退更简单架构。

二、原理拆解

- 常见模式包括主调度者+工作者、按类别路由专家、生成者+评审者、多候选并行表决以及对等 Agent handoff。
- A2A（Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务）解决 Agent 身份、能力发现、任务委派和结果交付；MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）解决 Agent 如何使用工具和数据。

三、实际执行流程

- 与单 Agent/固定 Workflow 建 baseline，比较端到端成功率、延迟、token/成本、工具错误、协调开销和稳定性。
- 还要评估 delegation accuracy、每个 Worker 成功率、消息质量、无效 handoff、并行收益和失败传播。
- 若准确率提升很小但成本翻倍，就应回退更简单架构。
- 可预先拆分、子任务可独立并行且需要不同上下文时，多 Agent 才容易产生收益。
- 任务高度共享状态、步骤少或工具调用成本很低时，单 Agent 往往更快、更容易调试。

四、边界、异常和权衡

- 任务合同应包含 task_id、目标、输入 schema（输入输出必须遵守的数据结构约定）、截止时间、预算、输出 schema 和错误语义。
- 上下文要按任务最小化分发，不应将用户全部历史和其他 Agent 的私有思考传给每个子 Agent。

五、形象例子

多 Agent 准确率提高 2% 但成本翻三倍，可能不值得上线。

八、Evals、Observability、Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）与安全

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；A2A：Agent-to-Agent 协议，用于智能体之间发现、委派和交付任务；schema：输入输出必须遵守的数据结构约定；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Eval、Trace、Guardrails 与运行治理

57. Agent Eval 与普通 LLM Eval 有什么不同？

一、直接回答

普通 LLM 评测往往只看一问一答；Agent（能够根据目标自主选择步骤和工具的软件智能体）还会调用工具、修改环境，所以必须多看三层：最后回答对不对、工具步骤合理不合理、环境里事情是否真的完成。同一个任务最好跑多次，因为模型路径会有随机性。

二、原理拆解

- Agent Eval（用数据集和指标评估系统能力）要区分 outcome 和 trajectory：最终答案可能正确，但中间调错工具、泄露数据或浪费巨额成本，这仍然是不合格的 Agent。
- Agent Eval 除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- 普通 LLM 评测往往只看一问一答；
- Agent 还会调用工具、修改环境，所以必须多看三层：最后回答对不对、工具步骤合理不合理、环境里事情是否真的完成。
- 同一个任务最好跑多次，因为模型路径会有随机性。
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

一个 Agent 最后找到了正确机票，但中间访问了不应访问的用户数据、重试了 30 次，结果虽对，轨迹仍不合格。

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；Eval：用数据集和指标评估系统能力；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

Agent Eval 分层



58. 什么是 trajectory evaluation?

一、直接回答

Trajectory

是一次运行的完整轨迹：模型消息、工具调用、参数、观察、状态变化和最终结果。可以检查必需工具是否调用、顺序是否合理、是否有多余步骤、参数是否正确，也可以用 LLM judge 比较参考轨迹。但正确路径可能不唯一，因此 outcome 检查通常比严格逐步完全匹配更可靠。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体） Eval（用数据集和指标评估系统能力）除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- Trajectory 是一次运行的完整轨迹：模型消息、工具调用、参数、观察、状态变化和最终结果。
- 可以检查必需工具是否调用、顺序是否合理、是否有多余步骤、参数是否正确，也可以用 LLM judge 比较参考轨迹。
- 但正确路径可能不唯一，因此 outcome 检查通常比严格逐步完全匹配更可靠。
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

轨迹可以检查 Agent 是否先查库存再下单，而不是跳过检查直接创建订单

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；Eval：用数据集和指标评估系统能力；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

59. 代码 grader、LLM-as-judge 和人工评审怎么组合？

一、直接回答

确定性结果优先代码

grader，例如测试是否通过、JSON（常用于系统之间传输结构化字段的文本格式）

schema（输入输出必须遵守的数据结构约定）、数据库状态；主观质量用带 rubric 的 LLM

judge，并定期与人工校准；高风险和边界样本保留人工评审。多层 grader

像瑞士奶酪模型，没有单一方法能覆盖所有错误。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体） Eval（用数据集和指标评估系统能力）除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- 确定性结果优先代码 grader，例如测试是否通过、JSON schema、数据库状态；
- 主观质量用带 rubric 的 LLM judge，并定期与人工校准；
- 高风险和边界样本保留人工评审。
- 多层 grader 像瑞士奶酪模型，没有单一方法能覆盖所有错误。
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

代码测试判断能否运行，LLM judge 判断文案质量，人审核高风险边界

术语复习

schema：输入输出必须遵守的数据结构约定；JSON：常用于系统之间传输结构化字段的文本格式；Agent：能够根据目标自主选择步骤和工具的软件智能体；Eval：用数据集和指标评估系统能力；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

60. Capability Eval 与 Regression Eval 有什么区别？

一、直接回答

Capability Eval（用数据集和指标评估系统能力）

选择当前较难任务，衡量能力上限，初始通过率可以较低；Regression Eval 保护已经会做的行为，期望接近

全通过。能力题一旦稳定可升级为回归题。实际失败、用户投诉和安全事件都应转成新的回归样本。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体） Eval
除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力） call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- Capability Eval 选择当前较难任务，衡量能力上限，初始通过率可以较低；
- Regression Eval 保护已经会做的行为，期望接近全通过。
- 能力题一旦稳定可升级为回归题。
- 实际失败、用户投诉和安全事件都应转成新的回归样本。
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）
需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

新能力题通过率从 30% 提到 60%；已经稳定的退款题则要求每次回归接近 100%

术语复习

Eval：用数据集和指标评估系统能力；Agent：能够根据目标自主选择步骤和工具的软件智能体；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

61. Agent 上线要记录哪些 Trace?

一、直接回答

至少有 Trace（一次请求从输入到所有调用和输出的完整记录）/task/user/thread ID、模型与 Prompt（发给模型的任务指令和上下文）版本、上下文摘要、每次工具名与参数脱敏、开始/结束时间、状态、错误、重试、token/成本和最终 outcome。多 Agent（能够根据目标自主选择步骤和工具的软件智能体）还需 parent-child span 与 handoff。敏感推理不必原样记录，但可保留结构化决策摘要和必要证据。

二、原理拆解

- Agent Eval（用数据集和指标评估系统能力）
除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace 应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- 至少有 Trace/task/user/thread ID、模型与 Prompt 版本、上下文摘要、每次工具名与参数脱敏、开始/结束时间、状态、错误、重试、token/成本和最终 outcome。
- 多 Agent 还需 parent-child span 与 handoff。
- 敏感推理不必原样记录，但可保留结构化决策摘要和必要证据。
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）
需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

一次请求要能看到模型、Prompt、工具、耗时、错误和最终数据库变化

术语复习

Prompt：发给模型的任务指令和上下文；Trace：一次请求从输入到所有调用和输出的完整记录；Agent：能够根据目标自主选择步骤和工具的软件智能体；Eval：用数据集和指标评估系统能力；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

62. Guardrails 应放在哪些层？

一、直接回答

输入层做越权、注入和内容分类；规划层限制目标和最大步数；工具层做 allowlist、参数校验、权限、速率和副作用分级；输出层做敏感信息与事实检查；环境层用沙箱和网络策略；高风险节点用 HITL。多层 guardrails（限制和校验 Agent（能够根据目标自主选择步骤和工具的软件智能体）输入、动作与输出的安全护栏）（限制和校验 Agent 输入、动作与输出的安全护栏）比单一系统 Prompt（发给模型的任务指令和上下文）更可靠。

二、原理拆解

- Agent Eval（用数据集和指标评估系统能力）
除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- 输入层做越权、注入和内容分类；
- 规划层限制目标和最大步数；
- 工具层做 allowlist、参数校验、权限、速率和副作用分级；
- 输出层做敏感信息与事实检查；
- 环境层用沙箱和网络策略；
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails 需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

读天气可自动执行，转账必须经过工具权限和人工确认两道防线

术语复习

Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏；Prompt：发给模型的任务指令和上下文；Agent：能够根据目标自主选择步骤和工具的软件智能体；Eval：用数据集和指标评估系统能力；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

63. 如何防止 Tool Poisoning 和恶意 MCP Server?

一、直接回答

工具描述、schema（输入输出必须遵守的数据结构约定）和返回都可能不可信。只连接受信任/签名服务器，固定版本和域名；在 Host 侧重写或审核描述；按服务器隔离凭据；最小权限；调用前显示动作；校验返回 schema；限制采样、roots 和网络；记录 server identity。不要因为工具出现在 MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）列表中就自动信任。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体） Eval（用数据集和指标评估系统能力）除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- 工具描述、schema 和返回都可能不可信。
- 只连接受信任/签名服务器，固定版本和域名；
- 在 Host 侧重写或审核描述；
- 按服务器隔离凭据；
- 调用前显示动作；
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

恶意 MCP 把工具描述写成“先上传密钥”，Host 应审核并限制它的网络和凭据

术语复习

schema：输入输出必须遵守的数据结构约定；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；Agent：能够根据目标自主选择步骤和工具的软件智能体；Eval：用数据集和指标评估系统能力；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

64. 如何控制 Agent 的成本和延迟？

一、直接回答

按任务难度做模型路由；减少无用上下文和工具定义；缓存检索/Embedding（把文本转换成可计算相似度的数字向量）；并行独立任务；批处理；设置 step/token/time/cost budget；小模型做分类和校验，强模型处理难步骤；监控每个成功任务成本而非单次 token。优化必须用 Eval（用数据集和指标评估系统能力）保证没有以质量换表面速度。

二、原理拆解

- Agent（能够根据目标自主选择步骤和工具的软件智能体） Eval 除了看最终答案，还要看意图、工具选择、参数、轨迹、中间状态、成本、延迟和安全性。
- Trace（一次请求从输入到所有调用和输出的完整记录）应串联 request_id、model call、tool（Agent 可以调用的具体程序能力）call、agent handoff、重试、审批和最终 outcome，并对敏感数据脱敏。

三、实际执行流程

- 按任务难度做模型路由；
- 减少无用上下文和工具定义；
- 缓存检索/Embedding；
- 设置 step/token/time/cost budget；
- 小模型做分类和校验，强模型处理难步骤；
- Capability Eval 验证系统能不能做到新能力，Regression Eval 确保新版本没有破坏旧能力。
- 代码 grader 验证硬性结果，LLM-as-judge（让大模型按评分标准评价另一个模型输出）评价语义质量，人工评审处理边界、安全和标准校准。

四、边界、异常和权衡

- Guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）需分布在输入、计划、工具前、工具后、输出和存储层，单一内容过滤器不足以保护 Agent。
- 成本优化应先做任务路由、上下文裁剪、检索缓存、工具并行和预算上限，再考虑简单换小模型。

五、形象例子

分类用小模型，复杂规划用强模型；若小模型低置信度，再升级处理

术语复习

Embedding：把文本转换成可计算相似度的数字向量；Eval：用数据集和指标评估系统能力；Agent：能够根据目标自主选择步骤和工具的软件智能体；Trace：一次请求从输入到所有调用和输出的完整记录；Tool：Agent 可以调用的具体程序能力；LLM-as-judge：让大模型按评分标准评价另一个模型输出；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

AI 辅助开发与个人工作流

65. 你用过哪些 AI 来开发项目？面试时怎么回答更稳妥？

一、直接回答

这道题最重要的是诚实分层。我会说自己真正高频使用哪些工具、主要用它们做什么，再说了解但不熟练的工具。例如我主要用 Codex 做跨文件实现和测试，用 Claude Code 做代码探索；Cursor、Copilot 如果只是了解，就明确说了解，不假装深度使用。

二、原理拆解

- 稳妥回答要区分“高频真实使用”、“在小任务中用过”和“了解但未实战”。面试官更关心你如何给 AI 任务、如何验收代码和遇到过什么失败，不是你能背出多少品牌。
- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 这道题最重要的是诚实分层。
- 我会说自己真正高频使用哪些工具、主要用它们做什么，再说了解但不熟练的工具。
- 例如我主要用 Codex 做跨文件实现和测试，用 Claude Code 做代码探索；
- Cursor、Copilot 如果只是了解，就明确说了解，不假装深度使用。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent（能够根据目标自主选择步骤和工具的软件智能体）
适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络和删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

可以说自己主要用 Codex 做跨文件实现，用 Claude Code 做代码探索，但只讲真实用过的

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

66. Codex、Claude Code、Cursor、GitHub Copilot 应如何比较？

一、直接回答

比较维度应是交互形态、长任务自主性、代码库上下文、工具/终端、权限模型、并行任务、IDE 集成、可扩展 Skills/MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）、模型选择、企业治理和成本，而不是简单排排名。终端/云 Coding Agent（能够根据目标自主选择步骤和工具的软件智能体）更适合跨文件长任务，IDE 助手更适合实时配对；团队常组合使用。具体能力变化快，应以实际评测为准。

二、原理拆解

- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 比较维度应是交互形态、长任务自主性、代码库上下文、工具/终端、权限模型、并行任务、IDE 集成、可扩展 Skills/MCP、模型选择、企业治理和成本，而不是简单排排名。
- 终端/云 Coding Agent 更适合跨文件长任务，IDE 助手更适合实时配对；
- 团队常组合使用。
- 具体能力变化快，应以实际评测为准。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent 适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络和删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

IDE 补全适合边写边改；长任务 Agent 适合夜间跑重构，但最终都要看 diff 和测试

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

67. 你的 AI 开发工作流是什么？

一、直接回答

我的流程不是把需求丢给 AI 后等答案。我会先写清验收标准，让 AI 只读代码并形成计划；确认后在隔离分支小步修改；每一步都跑测试和看 diff；复杂改动再让第二个 Agent（能够根据目标自主选择步骤和工具的软件智能体）独立 review；最后我自己检查架构、安全和是否真正满足需求。

二、原理拆解

- 一个可复述的流程是：人写目标/验收标准→AI 只读定位→AI 提修改计划→人确认→小步改动→自动测试和 diff→独立 review→人做最终责任判断。
- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 我的流程不是把需求丢给 AI 后等答案。
- 我会先写清验收标准，让 AI 只读代码并形成计划；
- 确认后在隔离分支小步修改；
- 每一步都跑测试和看 diff；
- 复杂改动再让第二个 Agent 独立 review；
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent 适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络和删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

我先让 AI 只读代码并提计划，确认后只改一个模块，跑完测试和看 diff，再让另一个 Agent 做独立 review，最后由我判断是否合并。

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

AI 辅助开发



68. 怎样给 Coding Agent 写一个高质量任务？

一、直接回答

包含业务目标、明确范围、不可修改项、相关入口、期望行为、验收命令、兼容性、安全限制和交付物。对复杂任务先要求探索与计划；对小任务直接给具体失败示例。避免只说“优化一下”，也不要把实现细节锁死到没有必要的程度。

二、原理拆解

- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 包含业务目标、明确范围、不可修改项、相关入口、期望行为、验收命令、兼容性、安全限制和交付物。
- 对复杂任务先要求探索与计划；
- 对小任务直接给具体失败示例。
- 避免只说“优化一下”，也不要把实现细节锁死到没有必要的程度。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent（能够根据目标自主选择步骤和工具的软件智能体）
适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络和删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

任务写明“只改 tools/collect.py，足球无结果时返回空列表，pytest 必须通过”

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

69. 什么时候让多个 Coding Agent 并行？

一、直接回答

适合互不重叠的模块、研究不同方案、独立测试/ 评审或大量同构任务。为每个 Agent（能够根据目标自主选择步骤和工具的软件智能体） 分配独立 worktree、明确文件范围和接口契约，最后由集成者合并。多个 Agent 同改同一核心文件会制造冲突；依赖链任务应串行或先定义稳定接口。

二、原理拆解

- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 适合互不重叠的模块、研究不同方案、独立测试/ 评审或大量同构任务。
- 为每个 Agent 分配独立 worktree、明确文件范围和接口契约，最后由集成者合并。
- 多个 Agent 同改同一核心文件会制造冲突；
- 依赖链任务应串行或先定义稳定接口。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent 适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络和删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

一个 Agent 改后端，另一个写测试，前提是接口先定好且使用独立 worktree

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

70. 如何验证 AI 生成代码，而不是盲目信任？

一、直接回答

检查 diff 和依赖；运行单元/集成/E2E、lint、type check、安全扫描；验证异常路径、权限和数据迁移；复现原问题；检查日志与性能；必要时让独立 Agent（能够根据目标自主选择步骤和工具的软件智能体）review，但最终以可重复的测试和真实环境 outcome 为准。AI 的自我声明“已完成”不是验收。

二、原理拆解

- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 检查 diff 和依赖；
- 运行单元/集成/E2E、lint、type check、安全扫描；
- 验证异常路径、权限和数据迁移；
- 检查日志与性能；
- 必要时让独立 Agent review，但最终以可重复的测试和真实环境 outcome 为准。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent 适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络 and 删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

AI 说修复完成后，我会复现原 bug、运行测试并检查它有没有顺手改坏别的文件

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

71. AI 开发中最常见的失败模式有哪些？

一、直接回答

误解需求、修改范围过大、编造不存在

API（软件之间按约定请求和返回数据的接口）、绕过测试、重复实现、把 mock

当真实、破坏现有改动、泄露密钥、只修表象、上下文过期。对应措施是小步提交、源码/官方文档核验、明确约束、强制测试、diff review、秘密扫描、失败日志和可回滚分支。

二、原理拆解

- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 误解需求、修改范围过大、编造不存在 API、绕过测试、重复实现、把 mock 当真实、破坏现有改动、泄露密钥、只修表象、上下文过期。
- 对应措施是小步提交、源码/官方文档核验、明确约束、强制测试、diff review、秘密扫描、失败日志和可回滚分支。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent（能够根据目标自主选择步骤和工具的软件智能体）
适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络 and 删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

AI 常编造不存在的方法，所以遇到陌生 API 我会查源码或官方文档

术语复习

API：软件之间按约定请求和返回数据的接口；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

72. AI 会不会让开发者基础能力退化？如何避免？

一、直接回答

如果只接受结果而不理解协议、数据流和失败原因，会退化；如果把 AI 用作探索、对照、自动化和评审，反而能扩大实践范围。保持能力的方法是能独立解释每个关键类/函数、亲自做架构和风险决策、阅读 diff、写验收、复盘故障，并定期不用 AI 完成基础练习。

二、原理拆解

- AI 开发工具的价值应按交互形态、长任务能力、代码库上下文、工具/权限、并行、可恢复和企业治理比较，而不是只按品牌排名。
- 回答使用经验要诚实区分高频实战、小任务试用和仅了解，重点是说明真实产出。

三、实际执行流程

- 如果只接受结果而不理解协议、数据流和失败原因，会退化；
- 如果把 AI 用作探索、对照、自动化和评审，反而能扩大实践范围。
- 保持能力的方法是能独立解释每个关键类/函数、亲自做架构和风险决策、阅读 diff、写验收、复盘故障，并定期不用 AI 完成基础练习。
- 人先定义需求和验收标准，AI 只读代码并提计划，人确认后小步实现，每步运行测试并查看 diff，最后独立 review。
- 并行 Agent（能够根据目标自主选择步骤和工具的软件智能体）
适合无重叠文件和稳定接口的任务，不适合多个 Agent 同时改同一核心文件。

四、边界、异常和权衡

- AI 生成代码必须看 diff、跑单测/集成测试、检查类型和异常路径，密钥、网络和删改操作要人工复核。
- 避免基础能力退化的方法是保留自己的架构判断、调试、测试和代码审查责任，不把 AI 当最终答案机。

五、形象例子

让 AI 写代码没问题，但自己必须能解释数据怎么流、错误为什么发生。
十、主流框架、架构选型与趋势判断

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

主流 Agent 框架、交互形态与生产平台

73. LangChain、LangGraph、AutoGen、CrewAI 的定位有什么不同？

一、直接回答

这四个框架有能力重叠，但关注点不同。LangChain（用于连接模型、提示词和工具的开发框架）偏模型、Prompt（发给模型的任务指令和上下文）、RAG（检索增强生成，先找资料再让模型基于资料回答）和工具组件连接；LangGraph（用状态图组织可暂停、可恢复 Agent（能够根据目标自主选择步骤和工具的软件智能体）流程的框架）偏带 State（流程在当前时刻保存的数据和执行状态）的图编排、持久化和恢复；AutoGen 偏多 Agent 对话和团队交互模式；CrewAI 偏用角色、任务和团队快速组织业务流程。我不会按流行度选，而是根据是否需要图状态、恢复、多 Agent 交互以及团队熟悉度来选。

二、原理拆解

- LangChain 主要提供模型、Prompt、Retriever 和 Tool（Agent 可以调用的具体程序能力）等可组合组件，适合搭建调用链和工具 Agent。
- LangGraph 把重点放在 State、Node（图流程中的一个处理步骤）、Edge（连接两个步骤并决定流向的边）、checkpoint（保存到某一步的任务状态快照）和持久化执行，适合可暂停、可恢复、有复杂分支的流程。
- AutoGen 强调多 Agent 对话和团队模式；CrewAI 强调 role、task、crew 和 process，更像按角色组织团队。

三、实际执行流程

- 只需模型+工具+RAG 组件时，可先用 LangChain。
- 需要循环、条件边、持久化、审批和恢复时，考虑 LangGraph。
- 需要多个 Agent 通过对话、Selector 或 Swarm 协作时，考虑 AutoGen。
- 需要用角色、任务和团队快速表达业务分工时，考虑 CrewAI。

四、边界、异常和权衡

- 四者存在能力重叠，不应用“谁更高级”排名，而应比较状态复杂度、恢复要求、团队协作模式和运维成本。

五、形象例子

需要模型和工具集成可用 LangChain；需要可恢复图执行选 LangGraph；多 Agent 群聊可看 AutoGen

术语复习

LangChain：用于连接模型、提示词和工具的开发框架；LangGraph：用状态图组织可暂停、可恢复 Agent 流程的框架；Prompt：发给模型的任务指令和上下文；State：流程在当前时刻保存的数据和执行状态；Agent：能够根据目标自主选择步骤和工具的软件智能体；RAG：检索增强生成，先找资料再让模型基于资料回答；Tool：Agent 可以调用的具体程序能力；checkpoint：保存到某一步的任务状态快照

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

74. OpenAI Agents SDK 常见核心概念有哪些？

一、直接回答

Agent（能够根据目标自主选择步骤和工具的软件智能体）/instructions、tools、handoffs、guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）、sessions/ memory、tracing 和 evals。常见模式包括 agent-as-Tool（Agent 可以调用的具体程序能力）、triage routing 和 handoff。它适合使用 OpenAI 模型生态快速搭建生产 Agent，但系统仍需要自行设计业务状态、权限、数据一致性和验收。

二、原理拆解

- OpenAI Agents SDK（封装好接口和工具的开发套件）的核心是 Agent、Tool、handoff、guardrails（限制和校验 Agent 输入、动作与输出的安全护栏）、session/memory、tracing 和 evals，分别对应决策者、动作、任务交付、安全、会话状态、链路记录和评估。

三、实际执行流程

- 定义 Agent 的指令和工具。
- 需要专家能力时用 handoff 把任务交给另一 Agent。
- 工具前后用 guardrails 做输入、参数和输出检查。
- 用 session/memory 保存会话状态，用 tracing 和 evals 检查质量。

四、边界、异常和权衡

- SDK 提供统一原语，但权限系统、业务幂等、数据治理和生产部署仍需项目自己设计。

五、形象例子

客服入口 Agent 判断类型后 handoff 给退款 Agent，并用 tracing 记录整条链

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；Tool：Agent 可以调用的具体程序能力；Guardrails：限制和校验 Agent 输入、动作与输出的安全护栏；SDK：封装好接口和工具的开发套件

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

75. AutoGen 的 RoundRobin、Selector、Swarm、GraphFlow 如何选择？

一、直接回答

RoundRobin 按固定顺序轮流，适合简单讨论；Selector 用模型或函数选择下一发言者；Swarm 通过 handoff 让专家局部转交；GraphFlow 用有向图表达顺序、并行、条件和循环。关键是设置 termination condition、max_turns 和状态保存，否则群聊容易空转。

二、原理拆解

- RoundRobin 按固定顺序轮流发言，Selector 动态选下一个 Agent（能够根据目标自主选择步骤和工具的软件智能体），Swarm 由 Agent 基于 handoff 对等转交，GraphFlow 用图明确控制分支和流向。

三、实际执行流程

- 固定头脑风暴可用 RoundRobin。
- 专家数量多、每轮需动态选人时用 Selector。
- 需要去中心化的能力转交时用 Swarm。
- 强流程合规、明确分支和可视化时用 GraphFlow。

四、边界、异常和权衡

- Selector 和 Swarm 更灵活，但更需要轮次上限、任务所有权和可观测性；否则容易来回踢球。

五、形象例子

固定轮流评审用 RoundRobin；动态选专家用 Selector；复杂依赖用 GraphFlow

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

76. Agentic Workflow 的五种经典模式是什么？

一、直接回答

常见归纳是 Prompt（发给模型的任务指令和上下文） Chaining（把复杂任务拆成多个前后衔接的提示步骤）（把复杂任务拆成多个前后衔接的提示步骤）（把复杂任务拆成多个前后衔接的提示步骤）（把复杂任务拆成多个前后衔接的提示步骤）（把复杂任务拆成多个前后衔接的提示步骤）、Routing、Parallelization、Orchestrator-Workers（主调度者拆任务、多个执行者并行完成的协作模式）（主调度者拆任务、多个执行者并行完成的协作模式）（主调度者拆任务、多个执行者并行完成的协作模式）（主调度者拆任务、多个执行者并行完成的协作模式）、Evaluator-Optimizer；在此基础上才是开放式 Autonomous Agent（能够根据目标自主选择步骤和工具的软件智能体）。面试时要说明适用条件：固定分解用 chaining，类别明确用 routing，可独立工作用 parallel，子任务未知用 orchestrator，质量可迭代度量用 evaluator。

二、原理拆解

- 五种经典 Agentic Workflow 模式通常是 Prompt Chaining、Routing、Parallelization、Orchestrator-Workers 和 Evaluator-Optimizer，分别解决分步、分类路由、并行、主从拆分和反馈改进。

三、实际执行流程

- 固定依赖的复杂生成用 Prompt Chaining。
- 不同输入走不同专家用 Routing。
- 互不依赖的子任务用 Parallelization。
- 任务需要运行时拆分用 Orchestrator-Workers。
- 需要多轮评审和修改用 Evaluator-Optimizer。

四、边界、异常和权衡

- 这些模式可以组合，但每增加一层循环或多 Agent 都会增加延迟、成本和失败面。

五、形象例子

文章先写后审是 Prompt Chaining；多个专家并行评审是 Parallelization

术语复习

Orchestrator-Workers：主调度者拆任务、多个执行者并行完成的协作模式；Prompt Chaining：把复杂任务拆成多个前后衔接的提示步骤；Prompt：发给模型的任务指令和上下文；Agent：能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

77. 什么是 Computer Use Agent? 与 API Tool Agent 有什么区别?

一、直接回答

Computer Use 通过截图/DOM和鼠标键盘操作 UI，适合没有 API（软件之间按约定请求和返回数据的接口）的应用，但更慢、更脆弱，且提示词注入和误操作风险更高。API Tool（Agent 可以调用的具体程序能力）参数结构化、可测试、权限清晰，能用 API 时应优先。Computer Use 需要视觉状态确认、动作后验证、敏感操作审批和隔离浏览器。

二、原理拆解

- Computer Use Agent（能够根据目标自主选择步骤和工具的软件智能体）（通过视觉、鼠标和键盘操作界面的智能体）通过截图或 DOM 理解界面，生成点击、输入和滚动动作；API Tool Agent 直接按 schema（输入输出必须遵守的数据结构约定）传参并读取结构化结果。

三、实际执行流程

- 有稳定 API 时优先 API Tool。
- 只有网页或桌面软件、无 API 时才用 Computer Use。
- Computer Use 每次动作后需重新观察界面并确认结果。
- 提交、删除、付款等高风险点击前需要人工审批。

四、边界、异常和权衡

- Computer Use 更慢、更脆弱，还可能误点；需要隔离浏览器、允许站点列表、动作回放和敏感操作审批。

五、形象例子

有 API 的订票应调 API；只有网页时才用 Computer Use 点击页面

术语复习

Tool: Agent 可以调用的具体程序能力；API: 软件之间按约定请求和返回数据的接口；Computer Use Agent: 通过视觉、鼠标和键盘操作界面的智能体；schema: 输入输出必须遵守的数据结构约定；Agent: 能够根据目标自主选择步骤和工具的软件智能体

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

78. 什么是 Model Routing?

一、直接回答

根据任务难度、模态、延迟、成本、隐私和工具能力选择不同模型。
可用规则、分类器或小模型路由，并设置强模型回退。路由要防止“简单任务误判导致失败”，因此需置信度、质量监控和定期 Eval（用数据集和指标评估系统能力）；缓存和 Prompt（发给模型的任务指令和上下文）/model 版本也要进入路由决策记录。

二、原理拆解

- Model Routing（按任务难度、成本和能力选择模型）把请求特征映射到模型：可考虑难度、模态、上下文长度、工具需求、延迟、成本、数据区域和安全级别。

三、实际执行流程

- 先用规则或小分类器判断任务类型。
- 简单分类、抽取和改写走低成本模型。
- 复杂规划、长上下文和高风险任务走更强模型。
- 低置信度、工具失败或质量校验不通过时升级模型。

四、边界、异常和权衡

- 路由规则也要版本化和评估，否则可能省了 token 却降低任务成功率。

五、形象例子

简单分类走便宜模型，遇到低置信度或复杂多步任务再切到强模型

术语复习

Prompt：发给模型的任务指令和上下文；Eval：用数据集和指标评估系统能力；Model Routing：按任务难度、成本和能力选择模型

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

79. 你怎么看 2026 年 Agent 开发的重点变化？

一、直接回答

现在的重点已经从“模型能不能调工具”转向“能不能长时间稳定工作”。面试更常追问状态能否恢复、上下文怎么控制、工具权限怎么限制、结果怎么评测，以及出错后能不能追踪。所以 Harness（为 Agent（能够根据目标自主选择步骤和工具的软件智能体）提供循环、工具、状态和约束的运行外壳）（为 Agent 提供循环、工具、状态和约束的运行外壳）、Context、Skills、Graph（用节点和连接关系表示任务流程的图结构）和 Evals 会越来越重要。

二、原理拆解

- 2026 年 Agent 开发的重点正从“模型能否调工具”转向“系统能否长时间稳定、可恢复、可评估、可审计地工作”，因此 Harness、Context、Skills、Graph 和 Evals 更受重视。

三、实际执行流程

- 开发时先定义完成标准和评估集。
- 为长任务建立状态、checkpoint（保存到某一步的任务状态快照）和恢复机制。
- 用 Skill（可复用的任务操作说明、脚本和配套资源）和 MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）把可复用能力与业务工具规范化。
- 对模型调用、工具轨迹、成本和安全做持续观测。

四、边界、异常和权衡

- 趋势题要区分稳定工程方向和短期产品热词，并用自己的项目需求说明为什么重要。

五、形象例子

过去关注“能不能调用工具”，现在面试更会追问能否恢复、评测、限权和追踪

术语复习

Harness：为 Agent 提供循环、工具、状态和约束的运行外壳；Graph：用节点和连接关系表示任务流程的图结构；Agent：能够根据目标自主选择步骤和工具的软件智能体；checkpoint：保存到某一步的任务状态快照；Skill：可复用的任务操作说明、脚本和配套资源；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

80. 请设计一个生产级 Agent 平台的分层架构。

一、直接回答

我会把生产级 Agent（能够根据目标自主选择步骤和工具的软件智能体）平台分为八层：入口与身份权限层、任务编排层、模型网关与路由层、工具/MCP（模型上下文协议，用于统一连接 Agent 与工具或数据）层、知识与记忆层、运行时与沙箱层、观测与评估层、治理与安全层。每层都要有清晰的输入输出合同、最小权限、失败语义和可观测字段，一次任务能通过唯一 request_id 从入口追踪到模型、工具、存储和最终输出。

二、原理拆解

- 平台可分为八层：入口/身份权限、任务编排、模型路由、工具与 MCP、知识/记忆、运行时/沙箱、观测/评估、治理/安全。
- 所有层不是自由调用，而是通过 task schema（输入输出必须遵守的数据结构约定）、tool（Agent 可以调用的具体程序能力）schema、state（流程在当前时刻保存的数据和执行状态）schema 和唯一 request_id 串起数据合同与追踪。

三、实际执行流程

- 入口层负责租户、身份、配额、请求规范化和敏感词筛查。
- 编排层管理 Loop、Graph（用节点和连接关系表示任务流程的图结构）、任务状态、暂停、恢复和人工审批。
- 模型网关负责模型选择、限流、超时、成本预算和失败切换。
- 工具层通过 MCP 统一注册、权限、参数校验、幂等和审计。
- 知识与记忆层管理 RAG（检索增强生成，先找资料再让模型基于资料回答）、短期状态、长期记忆、权限过滤和保留期。
- 运行时层提供队列、并发、沙箱、checkpoint（保存到某一步的任务状态快照）、重试、熔断和补偿。
- 观测和治理层统一 Trace（一次请求从输入到所有调用和输出的完整记录）、Eval（用数据集和指标评估系统能力）、费用、安全事件、数据脱敏、版本发布和回滚。

四、边界、异常和权衡

- 每层都必须有最小权限、超时、幂等键、错误语义和可观测字段，不能只靠中央 Agent 承担全部治理。
- 生产架构还要考虑多租户隔离、数据区域、灾备、容量计划和版本兼容。

五、形象例子

入口层像前台，Orchestration 像调度室，Tool 层像执行部门，Memory/RAG 像档案室，Observability 和 Governance 像监控与审计中心。

术语复习

Agent：能够根据目标自主选择步骤和工具的软件智能体；MCP：模型上下文协议，用于统一连接 Agent 与工具或数据；schema：输入输出必须遵守的数据结构约定；State：流程在当前时刻保存的数据和执行状态；Tool：Agent 可以调用的具体程序能力；Graph：用节点和连接关系表示任务流程的图结构；RAG：检索增强生成，先找资料再让模型基于资料回答；checkpoint：保存到某一步的任务状态快照

六、面试口述建议

面试时不需要一次把所有术语倒出来；先讲清原理和适用边界，面试官追问时再展开工程细节。

生产级 Agent 平台



延伸阅读方向

- OpenAI Agents SDK 文档: Agent、handoff、guardrails、tracing 与 evals
- LangGraph 文档: state graph、checkpoint、durable execution 和 human-in-the-loop
- Model Context Protocol 规范: tools、resources、prompts 与安全边界
- A2A Protocol 规范: Agent Card、task、message 和 artifact
- Microsoft AutoGen 文档: team、selector、swarm 与 graph flow